ОБ'ЄКТНО-ОРІЄНТОВАНЕ ПРОГРАМУВАННЯ · ЛЕКЦІЯ №10

Language Integrated Query (LINQ) у C# та платформі .NET 10

Повний академічний курс: Архітектура, синтаксис запитів, лямбда-вирази, стандартні оператори та моделі виконання

ФУНДАМЕНТ LINQ

Єдиний декларативний синтаксис

Запити до об'єктів у пам'яті, реляційних СКБД, XML та JSON

15 КАТЕГОРІЙ ОПЕРАТОРІВ

Повний спектр стандартних методів

Фільтрація, сортування, групування, реляційні з'єднання та агрегація

МОДЕЛІ ВИКОНАННЯ

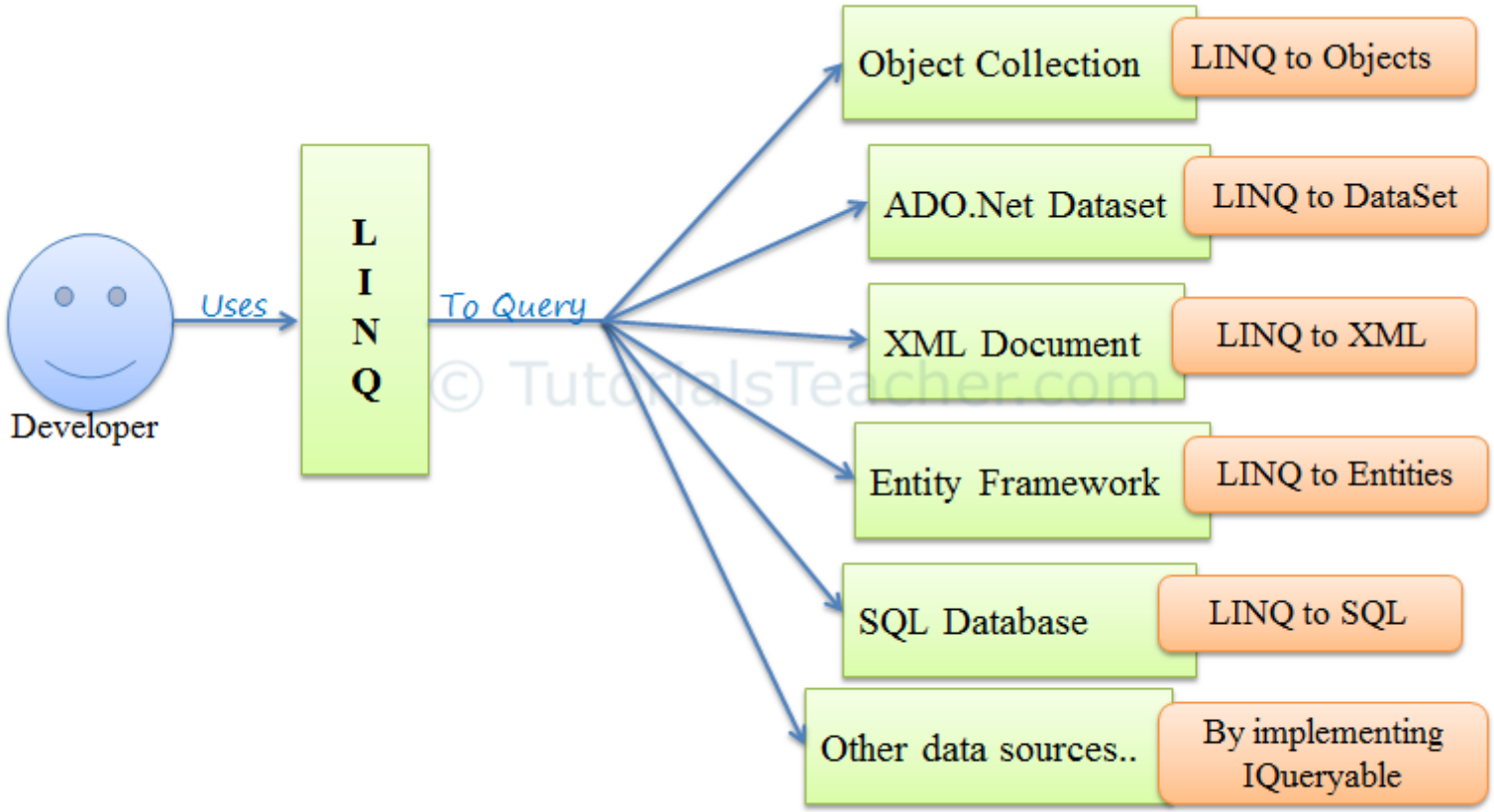
Оптимізація та продуктивність

Відкладене (Deferred) та негайне (Immediate) виконання, конвеєризація

Language-Integrated Query (LINQ) is a powerful query language introduced with .Net 3.5 & Visual Studio 2008. LINQ can be used with C# or Visual Basic to query different data sources.

LINQ (Language Integrated Query) is uniform query syntax in C# and VB.NET to retrieve data from different sources and formats. It is integrated in C# or VB, thereby eliminating the mismatch between programming languages and databases, as well as providing a single querying interface for different types of data sources.

For example, SQL is a Structured Query Language used to save and retrieve data from a database. In the same way, LINQ is a structured query syntax built in C# and VB.NET to retrieve data from different types of data sources such as collections, ADO.Net DataSet, XML Docs, web service and MS SQL Server and other databases.



LINQ Usage

LINQ queries return results as objects. It enables you to uses object-oriented approach on the result set and not to worry about transforming diffent formats of results into objects.



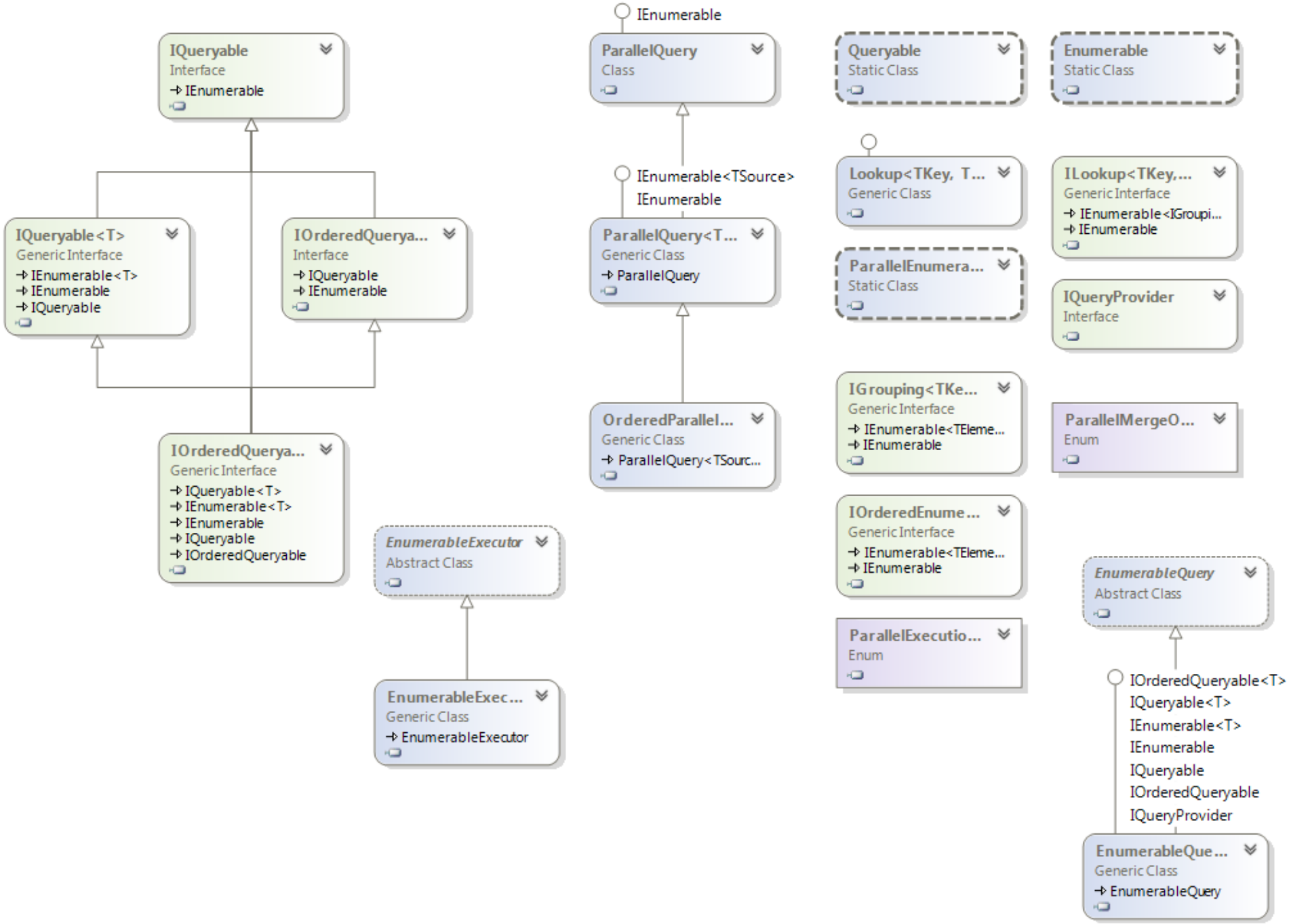
```
// Data source
string[] names = {"Bill", "Steve", "James", "Mohan" };

// LINQ Query
var myLinqQuery = from name in names
                  where name.Contains('a')
                  select name;

// Query execution
foreach(var name in myLinqQuery)
    Console.Write(name + " ");
```

Advantages of LINQ

- **Familiar language:** Developers don't have to learn a new query language for each type of data source or data format.
- **Less coding:** It reduces the amount of code to be written as compared with a more traditional approach.
- **Readable code:** LINQ makes the code more readable so other developers can easily understand and maintain it.
- **Standardized way of querying multiple data sources:** The same LINQ syntax can be used to query multiple data sources.
- **Compile time safety of queries:** It provides type checking of objects at compile time.
- **IntelliSense Support:** LINQ provides IntelliSense for generic collections.
- **Shaping data:** You can retrieve data in different shapes.

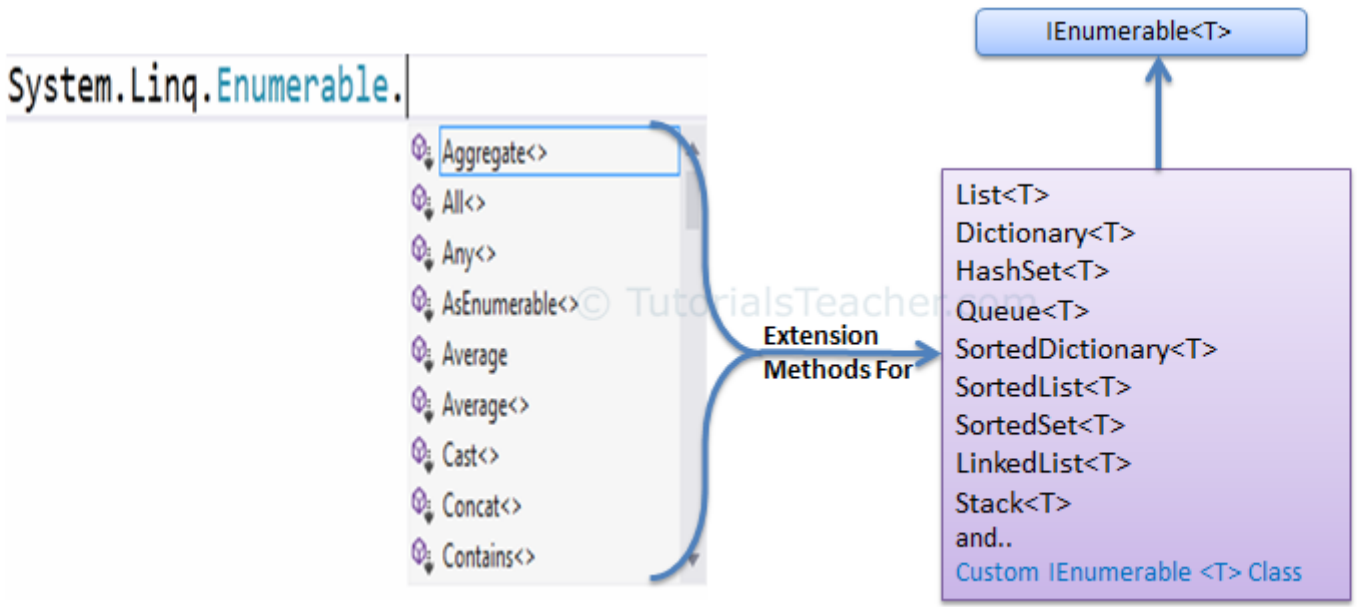

IQueryable
Interface
IEnumerable
IQueryable<T>
Generic Interface
IEnumerable<T>
IEnumerable
IQueryable
IOrderedQuerya...
Interface
IQueryable
IEnumerable
IOrderedQuerya...
Generic Interface
IQueryable<T>
IEnumerable<T>
IEnumerable
IQueryable
IOrderedQueryable
EnumerableExecutor
Abstract Class
EnumerableExec...
Generic Class
EnumerableExecutor
IEnumerable
ParallelQuery
Class
IEnumerable<TSource>
IEnumerable
ParallelQuery<T...
Generic Class
ParallelQuery
OrderedParallel...
Generic Class
ParallelQuery<TSourc...
Queryable
Static Class
Enumerable
Static Class
Lookup<TKey, T...
Generic Class
ILookup<TKey,...
Generic Interface
IEnumerable<IGroupi...
IEnumerable
ParallelEnumera...
Static Class
IQueryProvider
Interface
IGrouping<TKe...
Generic Interface
IEnumerable<TEleme...
IEnumerable
ParallelMergeO...
Enum
IOrderedEnume...
Generic Interface
IEnumerable<TEleme...
IEnumerable
ParallelExecutio...
Enum
EnumerableQuery
Abstract Class
IOrderedQueryable<T>
IQueryable<T>
IEnumerable<T>
IEnumerable
IQueryable
IOrderedQueryable
IQueryProvider
EnumerableQue...
Generic Class
EnumerableQuery

LINQ API

Enumerable

The Enumerable class includes extension methods for the classes that implement `IEnumerable<T>` interface, for example all the built-in collection classes implement `IEnumerable<T>` interface and so we can write LINQ queries to retrieve data from the built-in collections.

The following figure shows the extension methods included in `Enumerable` class that can be used with the generic collections in C# or VB.Net.

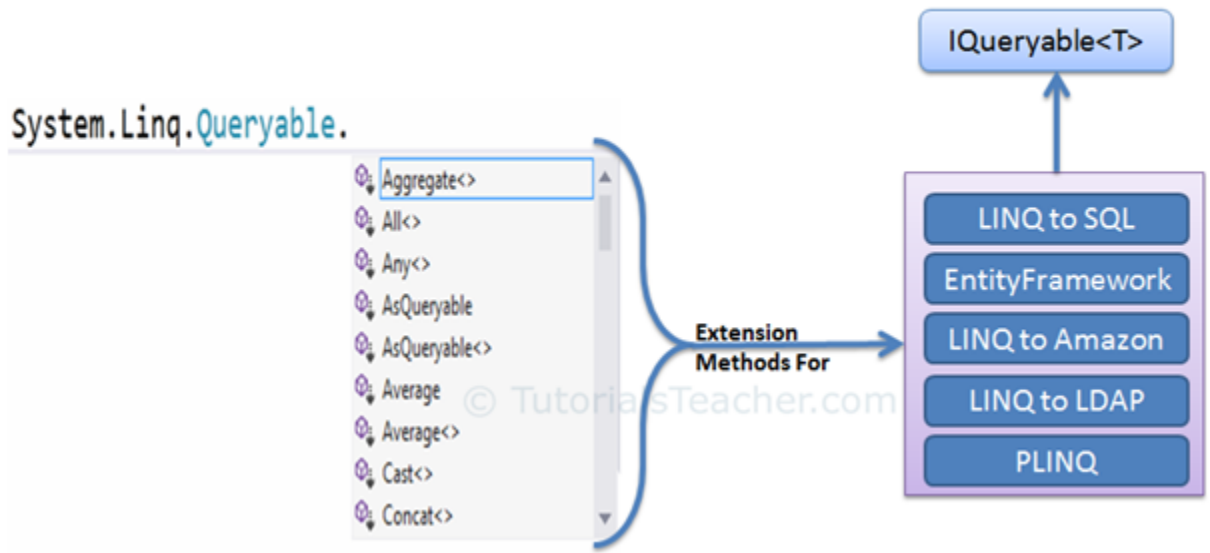


Queryable

The Queryable class includes extension methods for classes that implement IQueryable<t> interface. The `IQueryable<T>` interface is used to provide querying capabilities against a specific data source where the type of the data is known. For example, Entity Framework api implements `IQueryable<T>` interface to support LINQ queries with underlaying databases such as MS SQL Server.

Also, there are APIs available to access third party data; for example, LINQ to Amazon provides the ability to use LINQ with Amazon web services to search for books and other items. This can be achieved by implementing the `IQueryable` interface for Amazon.

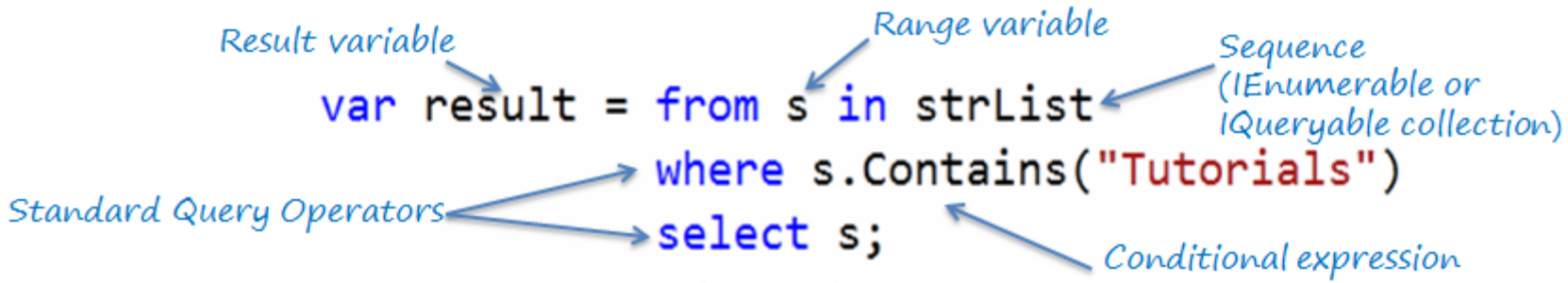
The following figure shows the extension methods available in the `Queryable` class can be used with various native or third party data providers.



LINQ Query Syntax

There are two basic ways to write a LINQ query to IEnumerable collection or IQueryable data sources

1. Query Syntax or Query Expression Syntax
2. Method Syntax or Method Extension Syntax or Fluent



LINQ Query Syntax

```
// Student collection
IList<Student> studentList = new List<Student>() {
        new Student() { StudentID = 1, StudentName = "John", Age = 13} ,
        new Student() { StudentID = 2, StudentName = "Moin",  Age = 21 } ,
        new Student() { StudentID = 3, StudentName = "Bill",  Age = 18 } ,
        new Student() { StudentID = 4, StudentName = "Ram" , Age = 20} ,
        new Student() { StudentID = 5, StudentName = "Ron" , Age = 15 }
    };

// LINQ Query Syntax to find out teenager students
var teenAgerStudent = from s in studentList
                      where s.Age > 12 && s.Age < 20
                      select s;
```

Points to Remember :

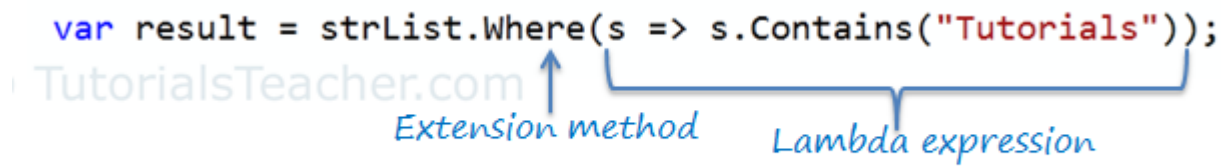
1) As name suggest, **Query Syntax** is same like SQL (Structure Query Language) syntax.

2) Query Syntax starts with *from* clause and can be end with *Select* or *GroupBy* clause.

3) Use various other opertors like filtering, joining, grouping, sorting operators to construct the desired result.

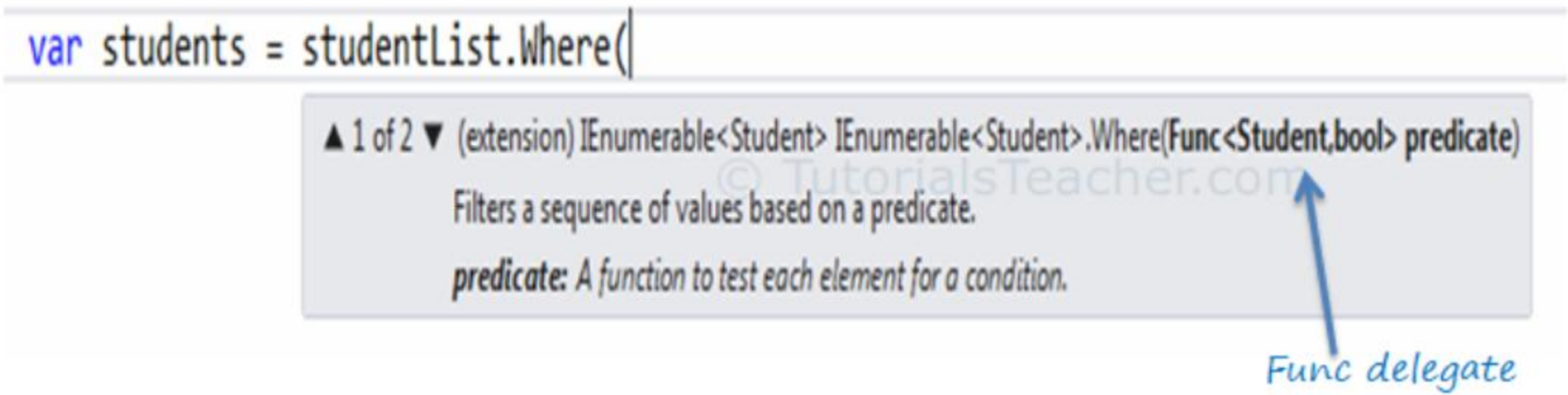
4) Implicitly typed variable - var can be used to hold the result of the LINQ query.

LINQ Method Syntax



LINQ Method Syntax Structure

If you check the signature of the Where extension method, you will find the Where method accepts a predicate delegate as Func<Student, bool>. This means you can pass any delegate function that accepts a Student object as an input parameter and returns a Boolean value as shown in the below figure. The lambda expression works as a delegate passed in the Where clause. Learn lambda expression in the next section.



Func delegate in Where

```
// Student collection
IList<Student> studentList = new List<Student>() {
        new Student() { StudentID = 1, StudentName = "John", Age = 13} ,
        new Student() { StudentID = 2, StudentName = "Moin",  Age = 21 } ,
        new Student() { StudentID = 3, StudentName = "Bill",  Age = 18 } ,
        new Student() { StudentID = 4, StudentName = "Ram" , Age = 20} ,
        new Student() { StudentID = 5, StudentName = "Ron" , Age = 15 }
    };

// LINQ Method Syntax to find out teenager students
var teenAgerStudents = studentList.Where(s => s.Age > 12 && s.Age < 20)
                                  .ToList<Student>();
```



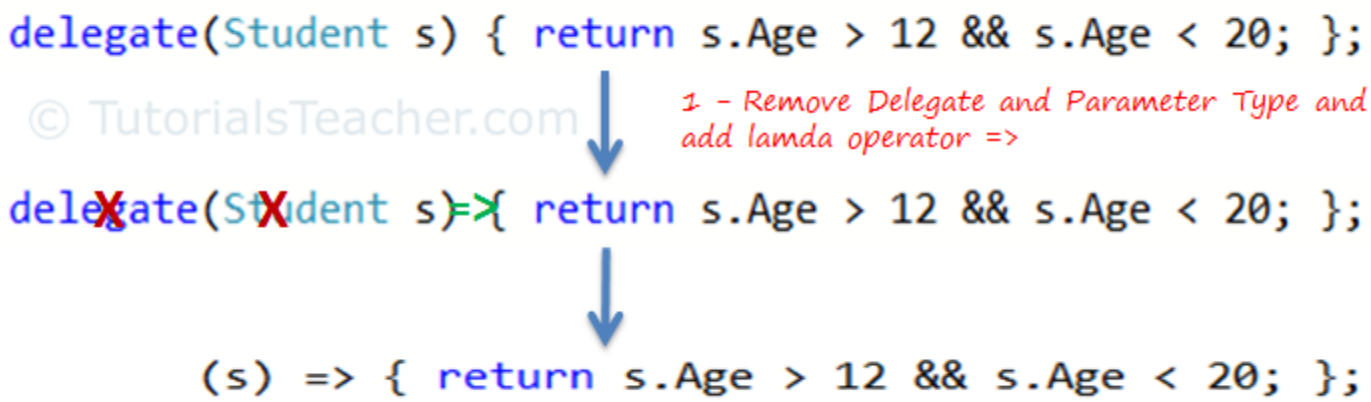
Points to Remember :

1) As name suggest, **Method Syntax** is like calling extension method.

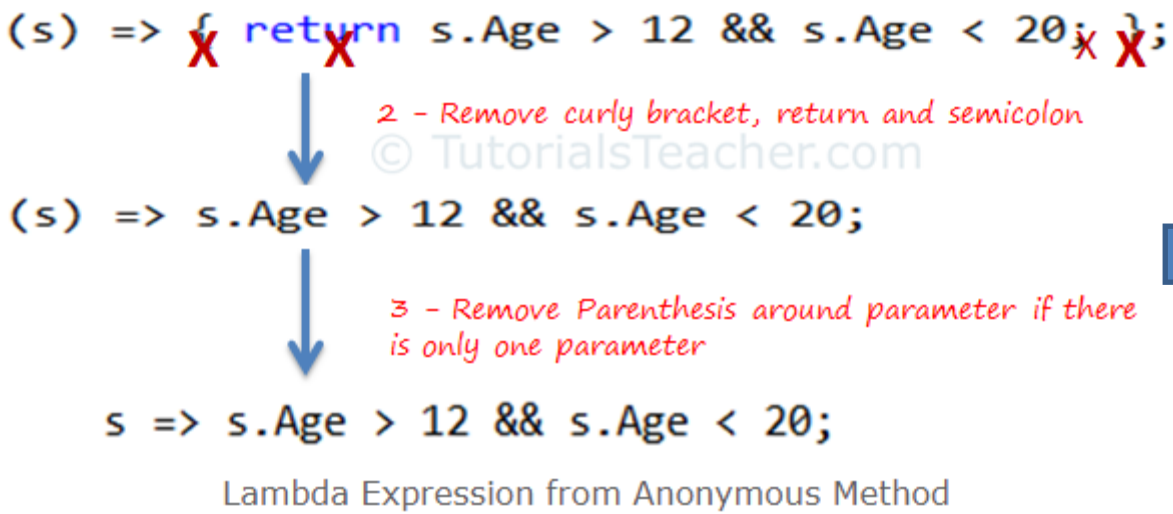
2) LINQ **Method Syntax** aka Fluent syntax because it allows series of extension methods call.

3) Implicitly typed variable - `var` can be used to hold the result of the LINQ query.

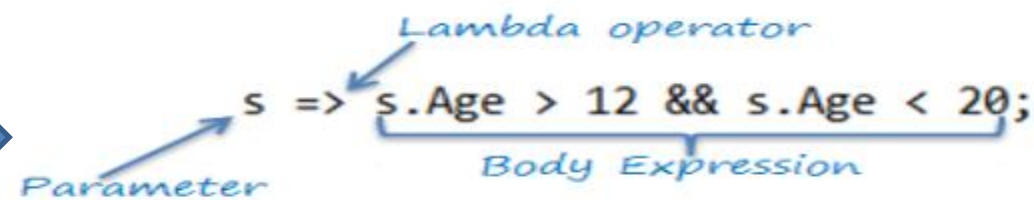
The Lambda expression evolves from anonymous method by first removing the delegate keyword and parameter type and adding a lambda operator =>.



Lambda Expression from Anonymous Method



Lambda Expression from Anonymous Method



Lambda Expression Structure in C#

Example: Specify Multiple Parameters in Lambda Expression C#

```
(s, youngAge) => s.Age >= youngage;
```

Example: Specify Parameter Type

```
(Student s,int youngAge) => s.Age >= youngage;
```

Example: Lambda Expression without Parameter

```
() => Console.WriteLine("Parameter less lambda expression")
```

Example: Multi Statements Lambda expression C#

```
(s, youngAge) =>
{
  Console.WriteLine("Lambda expression with multiple statements in the body");

  Return s.Age >= youngAge;
}
```

Example: Local Variable in Lambda expression C#

```
s =>
{
   int youngAge = 18;

    Console.WriteLine("Lambda expression with multiple statements in the body");

    return s.Age >= youngAge;
}
```

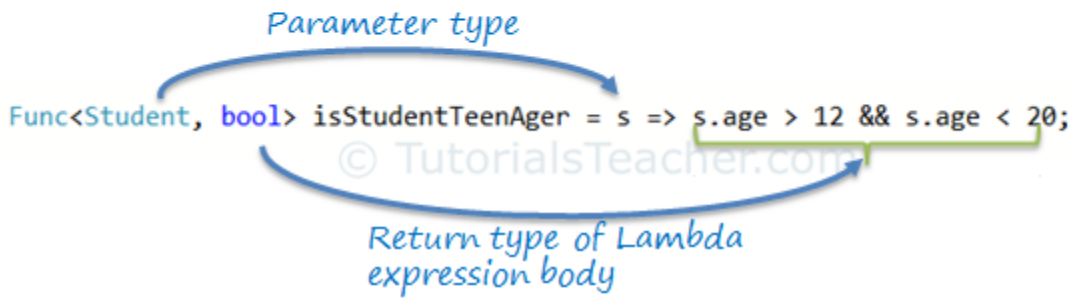
Example: Lambda Expression Assigned to Func Delegate C#

```
Func<Student, bool> isStudentTeenAger = s => s.age > 12 && s.age < 20;

Student std = new Student() { age = 21 };

bool isTeen = isStudentTeenAger(std);// returns false
```

In the above example, the Func delegate expects the first input parameter to be of Student type and the return type to be boolean. The lambda expression `s => s.age > 12 && s.age < 20` satisfies the Func<Student, bool> delegate requirement, as shown below:



Func delegate with Lambda Expression

The Func<> delegate shown above, would turn out to be a function as shown below.

```
bool isStudentTeenAger(Student s)
{
    return s.Age > 12 && s.Age < 20;
}
```

Example: Lamda Expression Assigned to Action Delegate C#

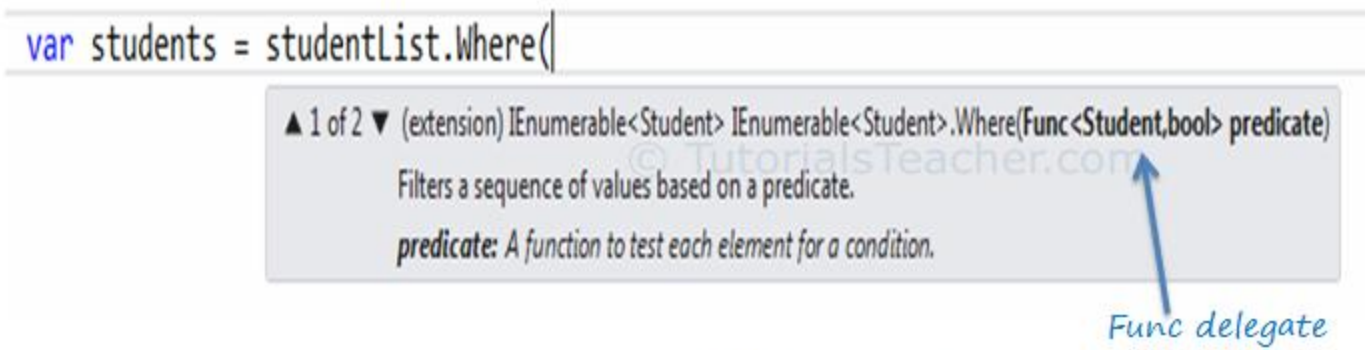
```
Action<Student> PrintStudentDetail = s => Console.WriteLine("Name: {0}, Age: {1} ", s.StudentName, s.Age);

Student std = new Student(){ StudentName = "Bill", Age=21};

PrintStudentDetail(std);//output: Name: Bill, Age: 21
```

Lambda Expression in LINQ Query

Usually lambda expression is used with LINQ query. Enumerable static class includes Where extension method for `IEnumerable<T>` that accepts `Func<TSource,bool>`. So, the Where() extension method for `IEnumerable<Student>` collection is required to pass `Func<Student,bool>`, as shown below:



Func delegate parameter in Where extension method

So now, you can pass the lambda expression assigned to the Func delegate to the Where() extension method in the method syntax as shown below:

Example: Func Delegate in LINQ Method Syntax

```
IList<Student> studentList = new List<Student>(){...};

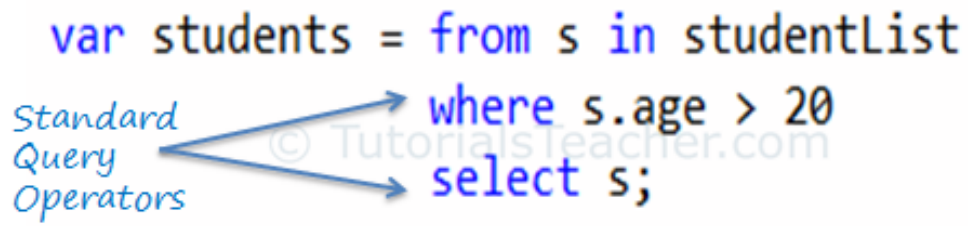
Func<Student, bool> isStudentTeenAger = s => s.age > 12 && s.age < 20;

var teenStudents = studentList.Where(isStudentTeenAger).ToList<Student>();
```

Points to Remember :

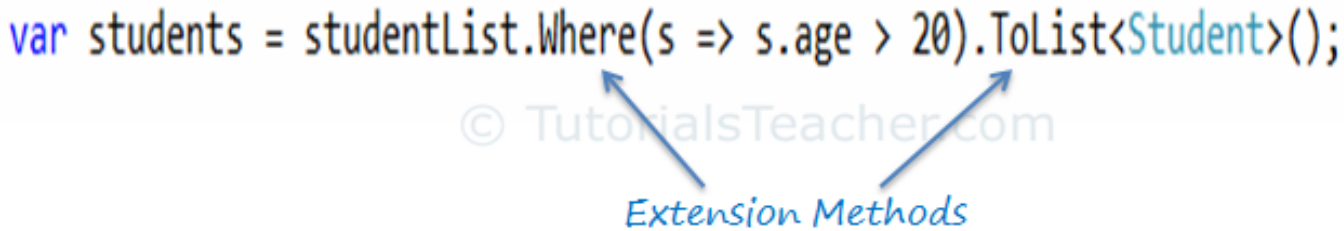
1) Lambda Expression is a shorter way of representing anonymous method.
2) Lambda Expression syntax: `parameters => body expression`
3) Lambda Expression can have zero parameter.
4) Lambda Expression can have multiple parameters in parenthesis ().
5) Lambda Expression can have multiple statements in body expression in curly brackets {}.
6) Lambda Expression can be assigned to Func, Action or Predicate delegate.
7) Lambda Expression can be invoked in a similar way to delegate.

Standard Query Operators in Query Syntax



Standard Query Operators in Query Syntax

Standard Query Operators in Method Syntax



Standard Query Operators in Method Syntax

Classification	Standard Query Operators
Filtering	Where, OfType
Sorting	OrderBy, OrderByDescending, ThenBy, ThenByDescending, Reverse
Grouping	GroupBy, ToLookup
Join	GroupJoin, Join
Projection	Select, SelectMany
Aggregation	Aggregate, Average, Count, LongCount, Max, Min, Sum
Quantifiers	All, Any, Contains
Elements	ElementAt, ElementAtOrDefault, First, FirstOrDefault, Last, LastOrDefault, Single, SingleOrDefault
Set	Distinct, Except, Intersect, Union
Partitioning	Skip, SkipWhile, Take, TakeWhile
Concatenation	Concat
Equality	SequenceEqual
Generation	DefaultEmpty, Empty, Range, Repeat
Conversion	AsEnumerable, AsQueryable, Cast, ToArray, ToDictionary, ToList

Що таке LINQ: Передумови виникнення та архітектура платформи

Революція декларативного підходу в .NET

- **Походження:** LINQ (Language Integrated Query) з'явився у .NET 3.5 / C# 3.0 як засіб уніфікованої роботи з даними.
- **Проблема парадигм (Impedance Mismatch):** Раніше для об'єктів писали імперативні цикли, для SQL — вразливі текстові рядки, для XML — XPath.
- **Єдина мова запитів:** LINQ інтегрував синтаксис запитів як першокласний елемент C# із повною перевіркою типів під час збірки проєкту.
- **Уніфікований конвеєр:** Один і той самий стиль запиту застосовується до пам'яті (Objects), баз даних (Entities) та документів (XML/JSON).

Архітектурна перевага:

Помилки в назвах властивостей або типах даних виявляються компілятором під час збірки, а не під час аварії у продакшені.

```
// 1. Чотири ключові провайдери платформи LINQ:
//    • LINQ to Objects (IEnumerable<T> у пам'яті CLR)
//    • LINQ to Entities / EF Core (IQueryable<T> -> SQL)
//    • LINQ to XML (XDocument, XElement, XAttribute)
//    • Parallel LINQ (PLINQ - паралельне виконання)

// 2. Декларативний запит C# до будь-якого джерела:
int[] numbers = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };

var evens = from n in numbers
            where n % 2 == 0
            orderby n descending
            select n;

// 3. Ітерація по конвеєру результатів:
foreach (var num in evens)
    Console.Write($"{num} ");

// -> Output: 10 8 6 4 2
```

Program.cs

Ключовий висновок:

LINQ інтегрував роботу з реляційними та об'єктними даними безпосередньо у мову C#, ліквідувавши розрив між кодом та СКБД.

LINQ to Objects: Робота з колекціями у пам'яті процесу

Концепція обробки даних у пам'яті CLR

- **Цільові джерела даних:** Будь-яка колекція, що реалізує інтерфейс IEnumerable<T> (масиви, List<T>, Dictionary<K,V>, HashSet<T>).
- **Простір імен System.Linq:** Вся функціональність надається статичним класом Enumerable через ланцюжки методів розширення.
- **Виконання в пам'яті процесу:** Запити виконуються процесором за допомогою скомпільованого IL-коду та делегатів Func<T>.
- **Відсутність парсингу:** Код запиту не передається як текст (на відміну від SQL чи regex) — він одразу виконується як нативний код .NET.

Pull-модель ітератора:

LINQ to Objects не створює нових колекцій наперед. Обчислення виконуються ліниво на вимогу споживача (on-demand via yield return).

```
// 1. Джерело даних у пам'яті процесу:
string[] names = { "Bill", "Steve", "James", "Mohan", "Jane" };

// 2. Декларативний запит (Query Syntax):
var myLinqQuery = from name in names
                  where name.Contains('a')
                  select name;

// 3. Еквівалентний методний запит (Method Syntax):
var methodQuery = names.Where(n => n.Contains('a'));

// 4. Потокове виконання під час ітерації:
foreach (var name in myLinqQuery)
    Console.WriteLine($"Знайдено: {name}");

// -> Output:
// -> Знайдено: James
// -> Знайдено: Mohan
// -> Знайдено: Jane
```

Program.cs

Архітектурна примітка:

Для роботи з LINQ to Objects достатньо підключити директиву using System.Linq; яка імпортує методи розширення Enumerable.

Практичний приклад: Перший запит LINQ to Objects

Анатомія простого запиту фільтрації

- **1. Джерело (Data Source):** Масив names реалізує IEnumerable<string>, виступаючи вхідним потоком для конвеєра ітератора.
- **2. Декларація запиту:** Оголошення запиту створює об'єкт конвеєра, але ще НЕ виконує фактичний перебір даних.
- **3. Виконання (Execution):** Фактична фільтрація викликається лише під час кроків циклу foreach (через GetEnumerator/MoveNext).
- **4. Стан пам'яті процесу:** До моменту ітерації жоден проміжний масив не створюється, мінімізуючи навантаження на збирач сміття (GC).

Відкладене виконання:

Якщо змінити масив names перед циклом foreach, запит автоматично поверне вже актуальні змінені дані!

```
// 1. Джерело даних у пам'яті процесу:
string[] names = { "Bill", "Steve", "James", "Mohan" };

// 2. Створення об'єкта LINQ-запиту:
var myLinqQuery = from name in names
                  where name.Contains('a')
                  select name;

// 3. Модифікація джерела перед ітерацією:
names[0] = "Dan"; // Bill -> Dan (Dan містить 'a'!)

// 4. Фактичне виконання під час ітерації:
Console.WriteLine("Результати відкладеного запиту:");
foreach (var name in myLinqQuery)
    Console.WriteLine($"-> {name}");

// -> Output:
// -> Dan
// -> James
// -> Mohan
```

Program.cs

Важливий момент:

Запит myLinqQuery зберігає лише інструкцію отримання даних, а не самі дані. Дані зчитуються 'на льоту' під час ітерації.

Дві гілки абстракції: IEnumerable проти IQueryable

IEnumerable<T> (LINQ to Objects)

- **Простір імен:** System.Collections.Generic, розширення в System.Linq.Enumerable.
- **Цільове середовище:** Колекції в оперативній пам'яті (In-Memory).
- **Параметри предикатів:** Скомпільовані делегати Func<T, bool> (байт-код IL).

```
// IEnumerable: виконання кодом процесора C#
IEnumerable<Student> list = studentList;
var res = list.Where(s => s.Age > 18);
// Дані вже завантажені в оперативну пам'ять!
```

IQueryable<T> (LINQ to SQL / Entities)

- **Простір імен:** System.Linq, розширення в System.Linq.Queryable.
- **Цільове середовище:** Віддалені бази даних (SQL Server, PostgreSQL, EF Core).
- **Параметри предикатів:** Дерева виразів Expression<Func<T, bool>> (AST).

```
// IQueryable: трансляція в чистий SQL провайдером
IQueryable<Student> query = dbContext.Students;
var res = query.Where(s => s.Age > 18);
// Генерує SQL: SELECT * FROM Students WHERE Age > 18
```

Головна відмінність:
IEnumerable<T> виконує код у пам'яті C#, а IQueryable<T> передає дерево синтаксису базі даних для генерації SQL.

Приклад у стилі Query Syntax (SQL-подібний)

Особливості синтаксису запитів у C#

- **Декларативність:** Описує, ЩО саме потрібно отримати з колекції, а не ЯК покроково перебирати масиви індексами.
- **Читабельність для розробників:** Знайомий синтаксис для тих, хто має досвід роботи з реляційними базами даних SQL.
- **Статична типізація:** Тип змінної s визначається компілятором автоматично з типу елементів колекції studentList.
- **Лямбда-децукрування:** Компілятор Roslyn автоматично перетворює цей вираз на Enumerable.Where(studentList, s => s.Age > 12 && s.Age < 20).

Порядок пропозицій:

from стоїть ПЕРШИМ (на відміну від SQL SELECT), щоб IDE одразу знала тип елементів для автодоповнення полів (IntelliSense).

```
// Колекція студентів:
IList<Student> studentList = new List<Student>() {
    new Student() { StudentID = 1, StudentName = "John", Age = 13 },
    new Student() { StudentID = 2, StudentName = "Moin", Age = 21 },
    new Student() { StudentID = 3, StudentName = "Bill", Age = 18 },
    new Student() { StudentID = 4, StudentName = "Ram",  Age = 20 },
    new Student() { StudentID = 5, StudentName = "Ron",  Age = 15 }
};

// Декларативний запит Query Syntax:
var teenAgerStudent = from s in studentList
                      where s.Age > 12 && s.Age < 20
                      select s;

foreach (var std in teenAgerStudent)
    Console.WriteLine($"Студент: {std.StudentName}, Вік: {std.Age}");

// -> Output: John (13), Bill (18), Ron (15)
```

Program.cs

Порада практики:

Query Syntax особливо зручний для запитів із кількома джерелами (join), групуванням (group by) та проміжними змінними (let).

Приклад у стилі Method Syntax (Fluent API)

Функціональний підхід через ланцюжки методів

- **Методи розширення (Extension Methods):** Методи Where, Select, OrderBy викликаються безпосередньо на екземплярі колекції через крапку.
- **Лямбда-вирази:** Умови фільтрації та проекції передаються у формі анонімних функцій s => s.Age > 12.
- **Композиція викликів (Fluent API):** Методи легко з'єднуються у ланцюжки: .Where(...).OrderBy(...).Select(...).
- **Повна потужність LINQ:** Усі 100% операторів LINQ доступні саме через Method Syntax (на відміну від Query Syntax).

Еквівалентність байт-коду:

Query Syntax компілюється у виклики Method Syntax. Швидкодія та результат виконання обох підходів АБСОЛЮТНО ІДЕНТИЧНІ.

```
// Колекція студентів:
IList<Student> studentList = new List<Student>() {
    new Student() { StudentID = 1, StudentName = "John", Age = 13
},
    new Student() { StudentID = 2, StudentName = "Moin", Age = 21
},
    new Student() { StudentID = 3, StudentName = "Bill", Age = 18
},
    new Student() { StudentID = 4, StudentName = "Ram",  Age = 20
},
    new Student() { StudentID = 5, StudentName = "Ron",  Age = 15 }
};

// Method Syntax (Fluent API ланцюжок):
var teenStudents = studentList.Where(s => s.Age > 12 && s.Age <
20);

foreach (var std in teenStudents)
    Console.WriteLine($"Студент: {std.StudentName}, Вік:
```

Промисловий стандарт:

У сучасному коді C# розробники частіше використовують саме Method Syntax завдяки його компактності та лаконічності.

Два стилі запитів: Query Syntax проти Method Syntax

Два синтаксичних підходи в мові C#

- **1. Query Syntax (Синтаксис запитів):** Декларативний підхід, синтаксично схожий на SQL (from ... in ... where ... select).
- **2. Method Syntax (Методний синтаксис):** Функціональний ланцюжок методів розширення з лямбда-виразами (.Where(...).Select(...)).
- **Компіляторна еквівалентність:** Компілятор C# Roslyn завжди транслює Query Syntax у виклики Method Syntax.
- **Універсальність бібліотеки BCL:** Будь-який запит можна записати у Method Syntax, але лише частину операторів підтримує Query Syntax.

Що обрати:

Query Syntax зручніший для Join, GroupJoin та let. Method Syntax є потужнішим і підтримує всі 100% операторів LINQ.

```
// 1. Декларативний Query Syntax (SQL-подібний стиль):
var q1 = from s in studentList
         where s.Age > 12 && s.Age < 20
         orderby s.StudentName
         select s.StudentName;

// 2. Функціональний Method Syntax (Fluent API ланцюжок):
var q2 = studentList
    .Where(s => s.Age > 12 && s.Age < 20)
    .OrderBy(s => s.StudentName)
    .Select(s => s.StudentName);

// 3. Ітерація та перевірка результату:
foreach (var name in q1)
    Console.WriteLine($"Підліток: {name}");

// -> Output:
// -> Підліток: Bill
// -> Підліток: John
// -> Підліток: Ron
```

Program.cs

Компіляторна магія:

Roslyn виконує Desugaring (децукрування), перетворюючи ключові слова from, where, select на виклики Where і Select.

Анатомія Query Syntax: Ключові слова та змінні діапазону

Складові частини виразу запиту

- **from s in studentList:** Оголошує змінну діапазону (range variable s) та вказує вхідне джерело даних.
- **where s.Age > 12 && s.Age < 20:** Логічний предикат фільтрації; пропускає лише елементи, для яких умова повертає true.
- **select s:** Фінальна проекція; визначає форму та тип результату, що повертається з конвеєра.
- **Типізація результату:** Запит повертає типізовану послідовність IEnumerable<Student> без необхідності ручного приведення.

Range Variable (Змінна діапазону):

Змінна s існує лише всередині запиту і позначає поточний ітерований елемент послідовності.

```
// Модель студентів:
IList<Student> studentList = new List<Student>() {
    new Student() { StudentID = 1, StudentName = "John", Age = 13 },
    new Student() { StudentID = 2, StudentName = "Moin", Age = 21 },
    new Student() { StudentID = 3, StudentName = "Bill", Age = 18 },
    new Student() { StudentID = 4, StudentName = "Ram",  Age = 20 },
    new Student() { StudentID = 5, StudentName = "Ron",  Age = 15 }
};

// Пошук підлітків (teenagers):
var teenAgerStudent = from s in studentList
                      where s.Age > 12 && s.Age < 20
                      select s;

foreach (var s in teenAgerStudent)
    Console.WriteLine($"{s.StudentName} ({s.Age} p.)");

// -> Output: John (13 p.), Bill (18 p.), Ron (15 p.)
```

Program.cs

Синтаксичний нюанс:

На відміну від SQL, де SELECT стоїть першим, у LINQ запит завжди починається з from, щоб IDE надавала автодоповнення (IntelliSense).

Ключові правила написання Query Syntax (Points to Remember)

Чотири золоті правила синтаксису запитів

- **1. Початок із from:** Будь-який запит у Query Syntax ОБОВ'ЯЗКОВО розпочинається з пропозиції from.
- **2. Завершення select або group:** Запит ОБОВ'ЯЗКОВО повинен закінчуватися або пропозицією select, або пропозицією group by.
- **3. Композиція операторів:** Між from та select можна довільно комбінувати where, orderby, join, let та додаткові from.
- **4. Неявна типізація var:** Для збереження результату запиту рекомендовано використовувати ключове слово var.

Завершальні конструкції:

select трансформує дані в IEnumerable<TResult>. group by розбиває дані на підколекції IEnumerable<IGrouping<TKey, TElement>>.

```
// Структурна схема Query Syntax:
// var result = from <range_var> in <data_source>
//              [where <condition>]
//              [orderby <key> [ascending|descending]]
//              [join ... on ... equals ...]
//              [let <var> = <expr>]
//              select <projection> | group <item> by <key>;

// Практичний приклад повної форми запиту:
var query = from s in studentList
            where s.Age >= 18
            orderby s.StudentName ascending
            select new { Name = s.StudentName, Age = s.Age };

foreach (var item in query)
    Console.WriteLine($"Студент: {item.Name}, Вік: {item.Age}");

// -> Output: Bill (18), Moin (21), Ram (20)
```

Program.cs

Чому var?

При проекції в анонімні типи ім'я типу генерується компілятором і невідоме заздалегідь, тому var є обов'язковим.

LINQ Method Syntax: Ланцюжки методів та лямбда-вирази

Анатомія методного синтаксису (Fluent API)

- **Виклики через крапку:** Method Syntax базується на викликах методів розширення, з'єднаних у ланцюжок (Fluent Interface).
- **Лямбда-делегати:** Кожен метод приймає функціональний делегат, який найчастіше описується лямбда-виразом s => ...
- **Сигнатура Where:** Метод Where приймає предикат Func<TSource, bool>, що повертає true/false для кожного елемента.
- **Автоматичне виведення типів:** Компілятор самостійно визначає generic-тип TSource, тому не потрібно писати .Where<Student>(...).

Func<Student, bool> у Where:

Лямбда s => s.Age > 12 автоматично приводиться компілятором до екземпляра делегата Func<Student, bool>.

```
// 1. Анатомія виклику Method Syntax:
var result = studentList.Where(s => s.Age > 12);
//           ^ джерело  ^ метод  ^ лямбда-предикат

// 2. Сигнатура методу розширення з простору Enumerable:
// public static IEnumerable<TSource> Where<TSource>(
//     this IEnumerable<TSource> source,
//     Func<TSource, bool> predicate);

// 3. Комбінування кількох операторів у ланцюжок:
var adultNames = studentList
    .Where(s => s.Age >= 18)
    .Select(s => s.StudentName);

foreach (var name in adultNames) Console.Write($"{name} ");

// -> Output: Moin Bill Ram
```

Program.cs

Fluent Syntax:

Method Syntax дозволяє комбінувати десятки методів: .Where(...).OrderBy(...).Select(...).Take(5).

Практика Method Syntax: Пошук студентів-підлітків

Фільтрація списку студентів через Where

- **Колекція студентів:** Список об'єктів Student із полями StudentID, StudentName та Age у пам'яті.
- **Виклик Where:** Метод фільтрує список за складеною умовою s.Age > 12 && s.Age < 20.
- **Результат:** Повертається лінивий перелічувач IEnumerable<Student>, що містить відповідні елементи.
- **Композиція умов:** Логічні оператори && (AND) та || (OR) підтримують повну логіку булевих виразів C#.

Points to Remember (Method Syntax):

1. Базується на методах розширення. 2. Має назву Fluent API. 3. Підтримує 100% операторів (включно з тими, яких немає в Query Syntax).

```
// Джерело даних:
IList<Student> studentList = new List<Student>() {
    new Student() { StudentID = 1, StudentName = "John", Age = 13 },
    new Student() { StudentID = 2, StudentName = "Moin", Age = 21 },
    new Student() { StudentID = 3, StudentName = "Bill", Age = 18 },
    new Student() { StudentID = 4, StudentName = "Ram",  Age = 20 },
    new Student() { StudentID = 5, StudentName = "Ron",  Age = 15 }
};

// Фільтрація підлітків (13-19 років) через Where:
var teenStudents = studentList.Where(s => s.Age > 12 && s.Age < 20);

foreach (Student std in teenStudents)
    Console.WriteLine($"Знайдено: {std.StudentName}, {std.Age} p.");

// -> Output:
// -> Знайдено: John, 13 p.
// -> Знайдено: Bill, 18 p.
// -> Знайдено: Ron, 15 p.
```

Program.cs

Перевага перед циклами:

Замість 8 рядків коду з foreach, if та add() у новий список, запит записується в один зрозумілий декларативний рядок.

Лямбда-вирази: Синтаксис, параметри та стрілочний оператор =>

Анатомія лямбда-виразу в C#

- **Вхідні параметри:** Зліва від оператора => записуються аргументи: s або (s, index). Типи виводяться автоматично.
- **Лямбда-оператор =>:** Читається як 'goes to' ('переходить у'). Відокремлює список аргументів від тіла виразу.
- **Тіло лямбди:** Справа від => розташовується вираз або блок коду: s.Age > 12 && s.Age < 20.
- **Контекст замикання (Closure):** Лямбда-вираз може захоплювати локальні змінні із зовнішнього лексичного контексту методу.

Еволюція синтаксису C#:

C# 1.0: Іменовані методи -> C# 2.0: Анонімні методи delegate(s) { return ...; } -> C# 3.0+: Лямбда-вирази s => ...

```
// Анатомія лямбда-виразу у фільтрації:
// (s)      =>       (s.Age > 12 && s.Age < 20)
//  ^       ^         ^
// парам.  'goes to'  тіло / повернене значення

// Приклад застосування з локальною змінною (Замикання):
int minAge = 18;
var adults = studentList.Where(s => s.Age >= minAge);

// Перебір відфільтрованих записів:
foreach (var s in adults)
    Console.WriteLine($"Повнолітній: {s.StudentName} ({s.Age})");

// -> Output:
// -> Повнолітній: Moin (21)
// -> Повнолітній: Bill (18)
// -> Повнолітній: Ram (20)
```

Program.cs

Компіляторна магія:

Компілятор самостійно виводить тип аргументу s як Student, оскільки він знає сигнатуру вхідної колекції.

Варіації синтаксису лямбда-виразів у C#

Параметри лямбда-виразів

- **Кілька параметрів:** Обов'язково беруться у круглі дужки: (s, youngAge) => ...
- **Явна типізація параметрів:** Якщо компілятор не може вивести тип: (Student s, int age) => ...
- **Без параметрів:** Порожні круглі дужки: () => Console.WriteLine(...)

```
// Кілька параметрів:
(s, youngAge) => s.Age >= youngAge;

// З явними типами:
(Student s, int youngAge) => s.Age >= youngAge;

// Без параметрів:
() => Console.WriteLine("Hello from Lambda!");
```

Тіло лямбда-виразу

- **Expression Lambda:** Один вираз, значення якого автоматично повертається (без return).
- **Statement Lambda:** Кілька інструкцій у фігурних дужках { ... }, вимагає явного return.
- **Побічні ефекти:** У Statement Lambda можна оголошувати локальні змінні та виклики логування.

```
// Statement Lambda з кількома рядками:
(s, youngAge) => {
    Console.WriteLine($"Перевірка {s.StudentName}");
    return s.Age >= youngAge;
};

// Expression Lambda (один рядок):
s => s.Age >= 18;
```

Рекомендація з чистоти коду:
У LINQ-запитах надавайте перевагу однорядковим Expression Lambdas — вони легко читаються і не мають побічних ефектів.

Призначення лямбда-виразів у делегат Func<T, bool>

Делегат Func як змінна першого класу

- **Узагальнений делегат Func:** Останній тип у кутових дужках Func<T1, T2, ..., TResult> завжди вказує тип повернення.
- **Func<Student, bool>:** Приймає об'єкт Student і повертає логічне значення bool (предикат перевірки).
- **Повторне використання:** Змінну-делегат можна зберегти, передавати як аргумент і викликати як звичайну функцію.
- **Інкапсуляція бізнес-правил:** Дозволяє централізовано визначати складні бізнес-правила в одному місці системи.

Сигнатура делегата:

Func<Student, bool> isStudentTeenAger = s => s.Age > 12 && s.Age < 20;

```
// 1. Оголошення делегата через лямбда-вираз:
Func<Student, bool> isStudentTeenAger =
    s => s.Age > 12 && s.Age < 20;

// 2. Створення тестових об'єктів студентів:
Student std1 = new Student() { StudentID = 1, StudentName = "Moin",
Age = 21 };
Student std2 = new Student() { StudentID = 2, StudentName = "Bill",
Age = 18 };

// 3. Прямий виклик збереженого делегата:
bool isMoinTeen = isStudentTeenAger(std1); // False
bool isBillTeen = isStudentTeenAger(std2); // True

Console.WriteLine($"Moin is teen: {isMoinTeen}");
Console.WriteLine($"Bill is teen: {isBillTeen}");

// -> Output:
// -> Moin is teen: False
// -> Bill is teen: True
```

Program.cs

Головна перевага:

Збереження складних бізнес-предикатів у делегати дозволяє повторно використовувати їх у різних LINQ-запитах без дублювання коду.

Еквівалентність Func делегата та іменованої функції

Трансляція лямбди компілятором під капотом

- **Вхідний тип:** Перший тип у списку узагальнень Func<Student, ...> задає тип вхідного параметра методу.
- **Вихідний тип:** Останній тип Func<..., bool> задає тип, що повертається інструкцією return.
- **Генерований метод:** Компілятор перетворює лямбду на звичайний приватний метод класу з атрибутом [CompilerGenerated].
- **Швидкодія:** Виклик через делегат не створює накладних витрат рефлексії і оптимізується JIT-компілятором.

Повна еквівалентність:

Делегат Func<Student, bool> під капотом є строго типізованим вказівником на функцію bool isStudentTeenAger(Student s).

```
// Форма 1. Лямбда-вираз через делегат Func:
Func<Student, bool> isTeenLambda =
    s => s.Age > 12 && s.Age < 20;

// Форма 2. Класичний іменований метод:
public bool IsTeenMethod(Student s)
{
    return s.Age > 12 && s.Age < 20;
}

// Форма 3. Передача іменованого методу в Where:
var teenList = studentList.Where(IsTeenMethod);

foreach (var s in teenList)
    Console.WriteLine($"Знайдено: {s.StudentName}");

// -> Output: John, Bill, Ron
```

Program.cs

Архітектурна перевага:

Лямбда-вирази дозволяють описувати локальну функціональну логіку прямо за місцем її використання без зайвого шаблонного коду.

Делегат Action<T> проти Func<T> та передача у Where

Делегат Action<T> (Повертає void)

- **Семантика:** Виконує дію, але НІЧОГО не повертає (аналог процедури).
- **Призначення:** Виведення на консоль, запис у лог, виклик події.
- **У LINQ:** Не використовується у Where/Select, бо не повертає результат.

```
// Action<Student>: дія без результату
Action<Student> PrintStudentDetail = s =>
    Console.WriteLine($"Name: {s.StudentName}, Age: {s.Age}");

var std = new Student() { StudentName = "Bill", Age = 21 };
PrintStudentDetail(std);
// Output: Name: Bill, Age: 21
```

Делегат Func<T, bool> (Повертає результат)

- **Семантика:** Обчислює значення і ОБОВ'ЯЗКОВО повертає його (чиста функція).
- **Призначення:** Предикати фільтрації, проекції, компаратори.
- **У LINQ:** Є основою методів Where, Select, OrderBy, Count тощо.

```
// Func<Student, bool>: предикат фільтра
Func<Student, bool> isTeen = s => s.Age < 20;

// Передача у метод розширення Where:
var result = studentList.Where(isTeen);
// Where приймає САМЕ Func<Student, bool>!
```

Золоте правило:
Усі фільтри та проекції LINQ працюють з Func<...>. Делегати Action<...> застосовуються для побічних дій (наприклад, .ForEach()).

Практика передачі делегата Func<Student, bool> у Where()

Чистий код без громіздких лямбда-рядків

- **Відокремлення логіки:** Складні бізнес-правила виносяться в окремі змінні делегатів або статичні методи.
- **Читабельність запиту:** Виклик studentList.Where(isStudentTeenAger) читається як звичайна людська фраза.
- **Легке тестування:** Делегат можна протестувати окремо модульними тестами Unit Tests без виконання запиту.
- **Динамічна підміна:** Делегат можна передавати як параметр методу, динамічно змінюючи критерій пошуку.

Еволюція коду:

1. Where(s => ...) -> 2. Func<Student, bool> isTeen = ... -> 3. Доменний предикат Student.IsTeenager

```
// 1. Оголошення предикатів фільтрації:
Func<Student, bool> isStudentTeenAger =
    s => s.Age > 12 && s.Age < 20;

Func<Student, bool> isHonorStudent =
    s => s.StudentID <= 3;

// 2. Використання збереженого делегата у Where:
var filteredStudents = studentList.Where(isStudentTeenAger);

// 3. Ітерація по відфільтрованій вибірці:
foreach (Student std in filteredStudents)
    Console.WriteLine($"Студент: {std.StudentName}, Вік: {std.Age}");

// -> Output:
// -> Студент: John, Вік: 13
// -> Студент: Bill, Вік: 18
// -> Студент: Ron, Вік: 15
```

Program.cs

Рекомендація з архітектури:

У корпоративних системах тримайте бізнес-предикати як статичні властивості моделей (наприклад, Student.IsTeenagerPredicate).

Стандартні оператори запитів: Query проти Method Syntax

Standard Query Operators у Query Syntax

- **Ключові слова C#:** from, where, select, group, into, orderby, join, let.
- **Трансляція:** Roslyn перетворює на виклики Where, Select, OrderByBy, GroupBy.
- **Читабельність:** Нагадує класичні SQL-запити до реляційних баз даних.

```
// Query Syntax:
var students = from s in studentList
               where s.Age > 20
               select s;

// Транслюється компілятором у Method Syntax!
```

Standard Query Operators у Method Syntax

- **Методи розширення:** Where, Select, OrderBy, GroupBy, Join, GroupJoin, Aggregate тощо.
- **Універсальність:** Підтримує всі оператори без винятку (понад 50 методів).
- **Функціональність:** Легко поєднується з іншими бібліотеками та розширеннями.

```
// Method Syntax:
var students = studentList
    .Where(s => s.Age > 20);

// Прямий виклик методу розширення Enumerable.Where
```

Еквівалентність:
Обидва синтаксиси є повністю взаємозамінними за швидкодією та результатом виконання.

Велика класифікація операторів LINQ (15 категорій)

Категорія операторів	Методи LINQ	Опис та призначення
`Filtering (Фільтрація)`	Where, OfType	Відбір елементів за умовою або типом
`Sorting (Сортування)`	OrderBy, OrderByDescending, ThenBy, ThenByDescending, Reverse	Впорядкування за зростанням або спаданням
`Grouping (Групування)`	GroupBy, ToLookup	Розбиття на групи за спільним ключем
`Joining (З'єднання)`	Join, GroupJoin	Реляційне зіставлення двох послідовностей
`Projection (Проекція)`	Select, SelectMany	Трансформація форми даних та сплющення 1:N
`Aggregation (Агрегація)`	Aggregate, Average, Count, LongCount, Max, Min, Sum	Обчислення скалярних підсумків та згортка
`Quantifiers (Квантори)`	All, Any, Contains	Логічна перевірка існування чи загальності
`Elements (Елементні)`	ElementAt, First, FirstOrDefault, Last, LastOrDefault, Single, SingleOrDefault	Вилучення конкретного одиничного елемента
`Set (Множини)`	Distinct, Except, Intersect, Union	Математичні операції над множинами без дублікатів
`Partitioning (Секціонування)`	Skip, SkipWhile, Take, TakeWhile, Chunk	Посторінкова пагінація та зрізи потоку
`Concatenation & Misc`	Concat, SequenceEqual, Zip	Послідовне об'єднання, перевірка рівності, зшивання
`Conversion & Execution`	AsEnumerable, AsQueryable, Cast, ToArray, ToList, ToDictionary	Матеріалізація в пам'ять та керування виконанням

Карта курсу:
Кожна з цих категорій детально розглядається на наступних слайдах із практичними прикладами C#.

Оператори фільтрації: Where та OfType

Метод LINQ	Опис	Що робить	Повертає	Приклад
`Where`	Фільтрує колекцію за заданою умовою (предикатом).	Пропускає елементи, для яких предикат повертає true.	IEnumerable<T>	`numbers.Where(x => x > 5)`
`OfType`	Фільтрує елементи колекції за їхньою сумісністю з типом T.	Відбирає лише ті елементи, які можна безпечно привести до типу T.	IEnumerable<TResult>	`list.OfType<string>()`

```
// 1. Where: перевірка логічної умови
var adults = studentList.Where(s => s.Age >= 18);

// 2. OfType: безпечна фільтрація за типом даних (ігнорує інші типи без помилок)
object[] mixed = { 1, "Hello", 3.14, "World", 42 };
var stringsOnly = mixed.OfType<string>(); // ["Hello", "World"]
```

Фізика роботи:
Where та OfType є потоковими операторами (Streaming, O(1) пам'яті). Вони не виділяють проміжних списків.

Оператор Where: Робота та сигнатура в .NET

Ключовий оператор фільтрації Enumerable.Where

- **Семантика Where:** Пропускає через конвеєр лише ті елементи, для яких вказаний предикат повертає значення true.
- **Предикат Func<TSource, bool>:** Чиста функція, що приймає поточний елемент колекції і повертає булевий результат.
- **Потокова обробка (Streaming):** Працює за принципом конвеєра: елементи не зберігаються в масиві, а віддаються один за одним.
- **Імутабельність джерела:** Початкова колекція ніколи не змінюється; генерується новий перелічувач результатів.

Відкладене виконання (Deferred Execution):

Фільтрація виконується тільки під час проходження циклу foreach або під час матеріалізації (ToList, ToArray).

```
// 1. Джерело даних:
IList<Student> studentList = new List<Student>() {
    new Student() { StudentID = 1, StudentName = "John", Age = 13 },
    new Student() { StudentID = 2, StudentName = "Moin", Age = 21 },
    new Student() { StudentID = 3, StudentName = "Bill", Age = 18 },
    new Student() { StudentID = 4, StudentName = "Ram",  Age = 20 },
    new Student() { StudentID = 5, StudentName = "Ron",  Age = 15 }
};

// 2. Фільтрація з кількома логічними умовами:
var filtered = studentList.Where(s => s.Age > 12 && s.Age < 20);

// 3. Ітерація по конвеєру:
foreach (var std in filtered)
    Console.WriteLine($"Знайдено: {std.StudentName}");

// -> Output: John, Bill, Ron
```

Program.cs

Оптимізація продуктивності:

Where не копіює дані. Якщо з мільйона записів підійдуть 5, у пам'яті будуть створені посилання тільки на ці 5 об'єктів.

Перевантаження Where: Фільтрація з використанням індексу

Сигнатура з порядковим номером елемента

- **Друге перевантаження Where:** Приймає делегат Func<TSource, int, bool>, де другий параметр — 0-базовий індекс елемента.
- **Автоматичний інкремент:** Індекс збільшується автоматично для кожного обробленого вхідного елемента під час перебору.
- **Тільки у Method Syntax:** Це перевантаження не має прямого синтаксичного аналога в Query Syntax (from ... where).
- **Практичні сценарії:** Відбір парних/непарних рядків, зеброве фарбування таблиць, обмеження перших N записів за позицією.

Сигнатура методу розширення:

```
public static IEnumerable<TSource> Where<TSource>(this IEnumerable<TSource> source, Func<TSource, int, bool> predicate);
```

```
// Вибірка елементів на парних індексах (0, 2, 4, ...):
var evenIndexStudents = studentList
    .Where((s, index) => index % 2 == 0);

foreach (var std in evenIndexStudents)
    Console.WriteLine($"Індексний студент: {std.StudentName}");
// Output: John (idx 0), Bill (idx 2), Ron (idx 4)

// Комбінування індексу та бізнес-властивості:
var complexFilter = studentList.Where((s, index) =>
    s.Age > 15 && index < 3);

foreach (var std in complexFilter)
    Console.WriteLine($"Фінал: {std.StudentName}");
// Output: Bill (Age 18, idx 2)
```

Program.cs

Важлива деталь:
Індекс передає позицію у вхідній послідовності, а не значення первинного ключа (StudentID) в базі даних!

Множинні пропозиції where у запитах C#

Множинні where у Query Syntax

- **Послідовний запис:** Можна вказувати кілька пропозицій where одну за одною.
- **Еквівалентність AND:** Елемент пройде далі лише тоді, коли задовольнить УСІ блоки where.
- **Читабельність:** Допомагає розбивати довгі громіздкі умови на окремі логічні рядки.

```
// Множинні where в одному запиті:
var filteredResult = from s in studentList
                     where s.Age > 12
                     where s.Age < 20
                     select s.StudentName;

// Еквівалентно: where s.Age > 12 && s.Age < 20
```

Ланцюжок .Where().Where() у Method Syntax

- **Композиція фільтрів:** Кожен наступний .Where() фільтрує результати попереднього.
- **Оптимізація BCL (Iterator Folding):** Сучасний .NET BCL об'єднує послідовні Where в один комбінований ітератор.
- **Динамічні фільтри:** Зручно додавати умови за прапорцями: if (filterAge) q = q.Where(...);

```
// Ланцюжок методів:
var result = studentList
    .Where(s => s.Age > 12)
    .Where(s => s.Age < 20);

// .NET оптимізує це в єдиний прохід!
```

Практична порада:
Використовуйте окремі where, коли умови належать до різних бізнес-правил (наприклад, перевірка статусу та діапазону дат).

Оператор OfType<T>: Безпечна фільтрація за типами

Робота з різнотипними колекціями та поліморфізмом

- **Суть OfType<T>:** Фільтрує колекцію, залишаючи лише ті елементи, які можна безпечно привести до типу T.
- **Безпека без помилок:** Елементи, що не відповідають цільовому типу, тихо пропускаються (без InvalidCastException).
- **Інструкція IL isinst:** На рівні байт-коду використовує інструкцію isinst, яка працює як оператор is у C#.
- **Підтримка наслідування:** Якщо вказати базовий тип OfType<Person>(), поверне як екземпляри Person, так і Student/Teacher.

OfType проти Cast:

Cast<T> намагається жорстко привести кожен елемент і кине виняток при невідповідності. OfType<T> просто відкидає зайві типи.

```
// Нетипізована колекція (наприклад, ArrayList або object[]):
IList mixedList = new ArrayList() {
    0,
    "One",
    "Two",
    3,
    new Student() { StudentID = 1, StudentName = "Bill" }
};

// 1. Отримати тільки рядки (ігноруючи числа та студентів):
var stringResult = mixedList.OfType<string>();
foreach (var str in stringResult) Console.Write($"{str} ");
// -> Output: One Two

// 2. Отримати тільки студентів:
var studentResult = mixedList.OfType<Student>();
foreach (var std in studentResult) Console.WriteLine(std.StudentName);
// -> Output: Bill
```

Program.cs

Сфера застосування:

OfType незамінний при міграції старого коду .NET 1.1/2.0 з ArrayList або при обробці ієрархій UI контролів (Controls.OfType<Button>()).

Практичний приклад застосування OfType у C#

Вибірка цілих чисел OfType<int>()

- **Джерело:** Змішаний список mixedList з числами, рядками та об'єктами.
- **Вибірка:** mixedList.OfType<int>() відбирає лише цілочисельні елементи.
- **Результат:** Повертає значення 0 та 3. Усі рядки та студенти проігноровані.

```
// Вибірка чисел:
var intResult = mixedList.OfType<int>();

foreach (int num in intResult)
    Console.Write(num + " ");

// Output: 0 3
```

Вибірка рядків OfType<string>()

- **Джерело:** Той самий змішаний список mixedList.
- **Вибірка:** mixedList.OfType<string>() відбирає текстові елементи.
- **Результат:** Повертає 'One' та 'Two'. Числа 0, 3 та об'єкти проігноровані.

```
// Вибірка рядків:
var strResult = mixedList.OfType<string>();

foreach (string s in strResult)
    Console.Write(s + " ");

// Output: One Two
```

Поліморфний бонус:
Якщо клас GraduateStudent успадковує Student, то list.OfType<Student>() поверне як звичайних студентів, так і випускників!

Оператори сортування: Впорядкування даних у LINQ

Концепція сортування послідовностей

- **Призначення:** Впорядковує елементи колекції у порядку зростання або спадання за одним або кількома ключами.
- **Стабільність сортування:** Усі оператори сортування у LINQ є стабільними (зберігають порядок однакових елементів).
- **Ключовий інтерфейс:** Методи сортування повертають спеціальний інтерфейс IOrderedEnumerable<T>.
- **Модель виконання:** Сортування є жадібним оператором з відкладеним стартом (буферизує всі елементи під час першого MoveNext).

Особливість виконання:

Для сортування неможливо віддати перший елемент без перегляду всієї колекції! Тому алгоритм QuickSort/TimSort зчитує всі дані в буфер.

```
// 1. Первинне сортування за віком (зростання):
var s1 = studentList.OrderBy(s => s.Age);

// 2. Первинне сортування за віком (спадання):
var s2 = studentList.OrderByDescending(s => s.Age);

// 3. Вторинне сортування (ThenBy / ThenByDescending):
var s3 = studentList
    .OrderBy(s => s.Age)
    .ThenBy(s => s.StudentName);

// 4. Виведення результату комплексного сортування:
foreach (var s in s3)
    Console.WriteLine($"{s.Age} p. - {s.StudentName}");

// -> Output:
// -> 13 p. - John
// -> 15 p. - Ron
// -> 18 p. - Bill
```

Program.cs

Продуктивність:

Уникайте сортування великих нескінченних послідовностей — операція вимагає читання всієї колекції у пам'ять!

Зведена таблиця операторів сортування в LINQ

Оператор	Query Syntax	Method Syntax	Опис дії	Тип повернення
`OrderBy`	orderby	`OrderBy()`	`Сортування за зростанням (Ascending)`	IOrderedEnumerable<T>
`OrderByDescending`	orderby ... desc	`OrderByDescending()`	`Сортування за спаданням (Descending)`	IOrderedEnumerable<T>
`ThenBy`	кома ',' у списку	`ThenBy()`	Вторинне сортування за зростанням	IOrderedEnumerable<T>
`ThenByDescending`	кома + desc	`ThenByDescending()`	Вторинне сортування за спаданням	IOrderedEnumerable<T>
`Reverse`	Не підтримується	`Reverse()`	Розгортає порядок елементів навпаки	IEnumerable<T>

```
// Приклад комбінованого сортування за віком, а для однакового віку - за ім'ям:
var sorted = studentList.OrderBy(s => s.Age).ThenBy(s => s.StudentName);
// Інверсія порядку відсортованих даних:
var reversed = sorted.Reverse();
foreach (var s in sorted)
    Console.WriteLine($"{s.Age} p. - {s.StudentName}");
// Output: 13 p. - John, 15 p. - Ron, 18 p. - Bill
```

SortingOperators.cs

Важливе правило:
ThenBy та ThenByDescending можуть викликатися ТІЛЬКИ після OrderBy/OrderByDescending, бо вони розширюють IOrderedEnumerable.

OrderBy та OrderByDescending у Query та Method Syntax

Первинне сортування колекцій

- **За замовчуванням (Ascending):** Сортування за зростанням: числа від меншого до більшого, текст від А до Я.
- **За спаданням (Descending):** OrderByDescending / descending: від більшого до меншого, текст від Я до А.
- **Кастомні компаратори:** Обидва методи приймають перевантаження з IComparer<TKey> для власної логіки порівняння.
- **Імутабельність джерела:** Початкова колекція залишається незмінною; генерується впорядкований ітератор.

Синтаксис у Query:

orderby s.StudentName (за замовчуванням ascending) або orderby s.StudentName descending (явне спадання)

```
// 1. Query Syntax (за спаданням):
var qAsc = from s in studentList
           orderby s.StudentName descending
           select s;

// 2. Method Syntax (за зростанням):
var mAsc = studentList.OrderBy(s => s.StudentName);

// 3. Ітерація по відсортованих даних:
foreach (var std in mAsc)
    Console.WriteLine($"Студент: {std.StudentName}");

// -> Output:
// -> Студент: Bill
// -> Студент: John
// -> Студент: Moin
// -> Студент: Ram
// -> Студент: Ron
```

Program.cs

Регістрозалежність:

За замовчуванням рядки сортуються з урахуванням поточної культури (CultureInfo.CurrentCulture).

Вторинне сортування: Оператори ThenBy та ThenByDescending

Багаторівневе сортування колекцій

- **Проблема повторного OrderBy:** Якщо викликати OrderBy двічі поспіль, друге сортування ПОВНІСТЮ ЗЛАМАЄ перше!
- **Призначення ThenBy:** Виконує сортування ТІЛЬКИ між елементами, у яких ключі попереднього сортування однакові.
- **Ланцюжки ThenBy:** Можна викликати необмежену кількість разів: .OrderBy().ThenBy().ThenByDescending().
- **Синтаксис запитів:** У Query Syntax вторинні ключі перераховуються через кому: orderby s.Age, s.StudentName.

Чому ламається OrderBy:

studentList.OrderBy(s => s.Age).OrderBy(s => s.StudentName) зламає сортування, пересортувавши все за ім'ям!

```
// 1. Правильне вторинне сортування через ThenBy:
var result = studentList
    .OrderBy(s => s.Age)
    .ThenBy(s => s.StudentName);

// 2. Аналог у Query Syntax через кому:
var query = from s in studentList
            orderby s.Age, s.StudentName
            select s;

// 3. Виведення результату багаторівневого сортування:
foreach (var s in result)
    Console.WriteLine($"{s.Age} p. - {s.StudentName}");

// -> Output:
// -> 13 p. - John
// -> 15 p. - Ron
// -> 18 p. - Bill
// -> 20 p. - Ram
// -> 21 p. - Moin
```

Program.cs

Типізація C#:

Метод ThenBy доступний лише для інтерфейсу IOrderedEnumerable<T>, тому компілятор не дозволить викликати його першим.

Оператор Reverse: Розгортання послідовності у зворотному напрямку

Інверсія порядку елементів

- **Суть оператора:** Повертає елементи вхідної колекції у прямо протилежному порядку без аналізу їх значення.
- **Тільки у Method Syntax:** У Query Syntax ключового слова reverse немає; використовується orderby descending.
- **LINQ проти List<T>.Reverse():** List<T>.Reverse() змінює список на місці (in-place void). LINQ Reverse() повертає новий IEnumerable<T>.
- **Буферизація:** Як і сортування, потребує зчитування елементів у внутрішній буфер для розгортання порядку.

Дві різні операції Reverse у C#:

list.Reverse() мутує список List<T>, тоді як LINQ list.AsEnumerable().Reverse() повертає новий незмінний ітератор.

```
// Джерело чисел:
int[] numbers = { 1, 2, 3, 4, 5 };

// Розгортання послідовності через LINQ:
var reversedNums = numbers.Reverse();

foreach (var n in reversedNums)
    Console.Write($"{n} ");
// -> Output: 5 4 3 2 1

// Розгортання після сортування:
var sortedDesc = studentList
    .OrderBy(s => s.Age)
    .Reverse();

foreach (var s in sortedDesc)
    Console.WriteLine($"{s.Age}: {s.StudentName}");
```

Program.cs

Чистота коду:

Використовуйте LINQ Reverse() для збереження незмінності (immutability) колекцій у функціональному стилі.

Вторинне сортування: ThenBy проти помилки повторного OrderBy

❌ Груба помилка: подвійний OrderBy

- **Суть проблеми:** Повторний виклик .OrderBy() створює НОВИЙ ітератор і ПОВНІСТЮ ЗНИЩУЄ попереднє сортування!
- **Наслідок:** Колекція буде відсортована ЛИШЕ за останнім полем. Попередній порядок буде втрачено.
- **Порушення контракту:** Зайві обчислення сортування QuickSort двічі.

```
// ❌ НЕПРАВИЛЬНО: OrderBy двічі ламає порядок!
var broken = studentList
    .OrderBy(s => s.Age)
    .OrderBy(s => s.StudentName);
// Попереднє сортування за Age СТЕРТО!
```

☑ Правильно: OrderBy + ThenBy

- **Суть рішення:** ThenBy створює складений компаратор у межах існуючого IOrderedEnumerable.
- **Збереження структури:** Вторинний ключ застосовується лише тоді, коли первинні ключі рівні.
- **Гнучкість напрямків:** ThenByDescending сортує вторинний ключ за спаданням.

```
//  ПРАВИЛЬНО: композитне сортування
var correct = studentList
    .OrderBy(s => s.Age)
    .ThenBy(s => s.StudentName);
// Ідеальне багаторівневе сортування!
```

Золоте правило архітектора:
У ланцюжку викликів OrderBy може бути лише ОДИН (на початку). Усі наступні рівні сортування — тільки ThenBy/ThenByDescending.

Порівняння: Оператор GroupBy проти ToLookup

Характеристика	GroupBy	ToLookup	Перевага	Сценарій використання
Тип виконання	Відкладене (Deferred Execution)	Негайне (Immediate Execution)	GroupBy не вантажить пам'ять заздалегідь	GroupBy для конвеєрів; ToLookup для кешу
Тип повернення	IEnumerable<IGrouping<TKey, TElement>>	ILookup<TKey, TElement>	ToLookup має індексатор lookup[key]	Швидкий багаторазовий пошук за ключем O(1)

```
// 1. GroupBy: Відкладене виконання (Deferred) - обчислюється при кожному foreach:
IEnumerable<IGrouping<int, Student>> grouped = studentList.GroupBy(s => s.Age);
foreach (var group in grouped)
    Console.WriteLine($"Ключ віку: {group.Key}, Кількість студентів: {group.Count()}");

// 2. ToLookup: Негайне виконання (Immediate) - створює постійний незмінний хеш-індекс 1:N у пам'яті:
ILookup<int, Student> lookup = studentList.ToLookup(s => s.Age);
var age18Students = lookup[18]; // Швидкий доступ O(1), lookup[99] безпечно поверне Empty без винятків!
```

GroupByVsToLookup.cs

Архітектурна різниця:
GroupBy повертає відкладений ітератор, а ToLookup негайно матеріалізує незмінний швидкий індекс у пам'яті.

Оператор GroupBy у Query Syntax (group ... by ...)

Синтаксис групування group s by s.Age

- **Завершальна пропозиція:** Запит у Query Syntax може завершуватися або select, або group by.
- **Структура IGrouping<TKey, TElement>:** Кожна група має властивість .Key та сама є колекцією IEnumerable<TElement>.
- **Проміжна змінна into:** Дозволяє продовжити запит після групування: group s by s.Age into g where g.Count() > 1...
- **Відкладене виконання:** Групування виконується ліниво під час перебору зовнішнього циклу foreach.

Дворівневий перебір:

Зовнішній цикл перебирає групи (foreach group in groupedResult). Внутрішній цикл перебирає елементи групи (foreach item in group).

```
// Query Syntax групування за віком:
var groupedResult = from s in studentList
                    group s by s.Age;

// Дворівневий перебір сформованих груп:
foreach (var ageGroup in groupedResult)
{
    Console.WriteLine($"Вікова група: {ageGroup.Key}");
    foreach (Student s in ageGroup)
    {
        Console.WriteLine($"  * {s.StudentName}");
    }
}

// -> Output:
// -> Вікова група: 13 -> John
// -> Вікова група: 21 -> Moin
// -> Вікова група: 18 -> Bill
```

Program.cs

Ключ групи (Key):

Ключем може бути як просте поле (Age), так і складний складений ключ: group s by new { s.Age, s.Department }.

Групування GroupBy у Method Syntax

Метод studentList.GroupBy(s => s.Age)

- **KeySelector лямбда:** s => s.Age вказує властивість, за якою розбиваються групи.
- **Збереження порядку:** GroupBy у LINQ гарантує збереження початкового відносного порядку появи груп та елементів.
- **Відкладена оцінка:** Групи формуються лише під час перебору в циклі foreach.
- **Проекція елементів групи:** GroupBy(s => s.Age, s => s.StudentName) дозволяє формувати групи лише з імен, уникаючи збереження всього об'єкта.

Можливості проекції груп:

GroupBy має перевантаження з вибором значення: GroupBy(s => s.Age, s => s.StudentName) для створення груп лише з імен.

```
// Method Syntax групування за віком:
var groupedResult = studentList.GroupBy(s => s.Age);

foreach (var ageGroup in groupedResult)
{
    Console.WriteLine($"AgeGroup: {ageGroup.Key} (К-сть:
{ageGroup.Count()})");
    foreach (Student s in ageGroup)
    {
        Console.WriteLine($"  - {s.StudentName}");
    }
}

// Проекція елементів групи лише в рядки імен:
var namesByAge = studentList.GroupBy(s => s.Age, s => s.StudentName);

// -> Output:
// -> AgeGroup: 13 (К-сть: 1) -> John
// -> AgeGroup: 21 (К-сть: 1) -> Moin
// -> AgeGroup: 18 (К-сть: 1) -> Bill
```

Program.cs

Хешування ключів:

Під капотом GroupBy будує хеш-таблицю відер (buckets) за значенням ключа, тому операція виконується за лінійний час O(N).

Оператор ToLookup: Негайний індекс 1:N у пам'яті

Незмінний словник-довідник ILookup<TKey, TElement>

- **Негайне виконання (Immediate):** На відміну від GroupBy, ToLookup матеріалізує структуру даних негайно при виклику.
- **Індексатор за ключем O(1):** lookup[18] миттєво повертає всіх студентів віком 18 років за константний час O(1).
- **Безпека відсутності ключа:** Якщо ключ не знайдено, повертає порожню послідовність Enumerable.Empty<T> (без винятків!).
- **Виключно в Method Syntax:** ToLookup підтримується лише у Method Syntax, у Query Syntax його аналога немає.

Сценарій використання:

Ідеально підходить для створення довгоживучого незмінного кешу для швидкого багаторазового пошуку списків за ключем.

```
// Негайне створення індексу Lookup:
var lookupResult = studentList.ToLookup(s => s.Age);

// 1. Швидкий прямий доступ до студентів віком 18 років:
foreach (var s in lookupResult[18])
    Console.WriteLine(s.StudentName); // John, Bill

// 2. Безпечний доступ до відсутнього ключа 99 (не кидає помилки!):
var emptyGroup = lookupResult[99];
Console.WriteLine(emptyGroup.Count()); // Output: 0

// 3. Повний перебір груп, як у GroupBy:
foreach (var group in lookupResult)
    Console.WriteLine($"Group: {group.Key}, Count:
{group.Count()}");

// -> Output: Group 13: 1, Group 21: 1, Group 18: 1
```

Program.cs

Головна перевага:

ToLookup є безпечною версією Dictionary<TKey, List<TElement>>, оптимізованою для зв'язків один-до-багатьох.

Оператори реляційного з'єднання: Join та GroupJoin

Метод LINQ	Опис / Призначення	Що робить	Аналог у SQL	Приклад	Результат
`Join`	Об'єднує дві колекції за спільним ключем і повертає послідовність пар елементів.	Виконує внутрішнє об'єднання (inner join): бере лише ті елементи, для яких є збіг в обох колекціях.	INNER JOIN	`students.Join(standards, s => s.StandardID, st => st.StandardID, (s, st) => new { s.Name, st.Name })`	Список студентів разом із назвами їхніх класів
`GroupJoin`	Об'єднує дві колекції за ключем, але повертає групи пов'язаних елементів (1:N).	Створює колекцію груп, де кожен батьківський елемент містить підколекцію дочірніх.	LEFT OUTER JOIN	`standards.GroupJoin(students, st => st.StandardID, s => s.StandardID, (st,`	Кожен клас зі списком усіх своїх студентів

```
// 1. Внутрішнє з'єднання Join (Inner Join): плаский список пар студент-клас
var inner = studentList.Join(standardList, s => s.StandardID, st => st.StandardID,
    (s, st) => new { s.StudentName, st.StandardName });
foreach (var item in inner) Console.WriteLine($"{item.StudentName} -> {item.StandardName}");

// 2. Групове з'єднання GroupJoin (Ієрархічне 1:N): кожен клас містить список студентів
var groups = standardList.GroupJoin(studentList, st => st.StandardID, s => s.StandardID,
    (st, stds) => new { st.StandardName, Students = stds });
foreach (var g in groups) Console.WriteLine($"{g.StandardName}: {g.Students.Count()} учнів");
```

JoinVsGroupJoin.cs

Різниця структур:
Join створює плаский денормалізований список рядків, тоді як GroupJoin створює нормалізоване ієрархічне дерево 1:N.

Практичний приклад: Внутрішнє з'єднання двох списків рядків

Зіставлення рядкових колекцій за спільними значеннями

- **Колекція 1 (Outer):** strList1 містить елементи: 'One', 'Two', 'Three', 'Four'.
- **Колекція 2 (Inner):** strList2 містить елементи: 'One', 'Two', 'Five', 'Six'.
- **З'єднання Join:** Зіставляє елементи обох списків за рівністю рядків str1 == str2.
- **Хешування ключів:** Для strList2 будується швидка хеш-таблиця у пам'яті для миттєвого пошуку O(1).

Результат Inner Join:

Повертає лише спільні елементи: 'One' та 'Two'. Усі незбіжні елементи ('Three', 'Four', 'Five', 'Six') відкидаються.

```
// Дві вхідні колекції:
IList<string> strList1 = new List<string>() {
    "One", "Two", "Three", "Four"
};
IList<string> strList2 = new List<string>() {
    "One", "Two", "Five", "Six"
};

// Внутрішнє об'єднання за рівністю рядків:
var innerJoin = strList1.Join(
    strList2,               // внутрішня колекція
    str1 => str1,           // ключ зовнішньої
    str2 => str2,           // ключ внутрішньої
    (str1, str2) => str1);  // формування результату

foreach (var str in innerJoin)
    Console.WriteLine($"Спільний елемент: {str}");

// -> Output:
// -> Спільний елемент: One
// -> Спільний елемент: Two
```

Program.cs

Складність Hash Join:

LINQ будує внутрішню хеш-таблицю для strList2, тому пошук спільних елементів виконується за лінійний час O(N + M).

Реляційні класи Student і Standard та з'єднання сутностей

Модель реляційних даних у C#

- **Клас Student:** Містить поля StudentID, StudentName та зовнішній ключ StandardID (посилання на клас).
- **Клас Standard:** Містить первинний ключ StandardID та текстову назву класу StandardName.
- **Реляційний зв'язок:** Кожен студент прив'язаний до певного стандарту через зовнішній ключ StandardID.
- **Проекція зв'язку:** Join дозволяє з'єднати сутності та створити анонімний об'єкт із полями з обох таблиць.

З'єднання сутностей:

З'єднуємо таблицю студентів із таблицею стандартів за рівністю student.StandardID == standard.StandardID.

```
// З'єднання сутностей студентів та стандартів:
var innerJoin = studentList.Join(
    standardList,
    student => student.StandardID,
    standard => standard.StandardID,
    (student, standard) => new {
        StudentName = student.StudentName,
        StandardName = standard.StandardName
    });

// Виведення результатів з'єднання:
foreach (var obj in innerJoin)
{
    Console.WriteLine($"{obj.StudentName} навчається у
{obj.StandardName}");
}

// -> Output:
// -> John навчається у Standard 1
// -> Moin навчається у Standard 1
// -> Bill навчається у Standard 2
```

Program.cs

Результат проекції:

Отримуємо кортежі: John -> Standard 1, Moin -> Standard 1, Bill -> Standard 2, Ram -> Standard 2.

Анатомія параметрів оператора Join у Method Syntax

П'ять ключових компонентів методу Join

- **1. Outer Sequence (Зовнішня):** Колекція, на якій викликається метод (studentList).
- **2. Inner Sequence (Внутрішня):** Друга колекція, що передається першим аргументом (standardList).
- **3. Key Selectors:** Дві лямбди вибору ключів: student => student.StandardID та standard => standard.StandardID.
- **4. Result Selector:** Функція формування об'єкта результату (student, standard) => new { ... }.

Правило оптимізації Hash Join:

LINQ будує хеш-таблицю для Inner Sequence. Рекомендується передавати меншу за обсягом таблицю як Inner Sequence!

```
// Анатомія параметрів методу Join:
var joinResult = studentList.Join(
    standardList,                    // 1. Inner Sequence
    student => student.StandardID,   // 2. Outer Key Selector
    standard => standard.StandardID,// 3. Inner Key Selector
    (student, standard) => new {     // 4. Result Selector
        StudentFullName = student.StudentName,
        StandardFullName = standard.StandardName
    });

foreach (var item in joinResult)
    Console.WriteLine($"{item.StudentFullName} ->
{item.StandardFullName}");

// -> Output:
// -> John -> Standard 1
// -> Moin -> Standard 1
// -> Bill -> Standard 2
// -> Ram -> Standard 2
```

Program.cs

Безпека типів:

Типи ключів OuterKey та InnerKey повинні точно збігатися (наприклад, обидва мають бути int).

Оператор Join у Query Syntax (join ... on ... equals ...)

Синтаксис реляційного з'єднання запитів

- **join <var> in <col>:** Оголошує внутрішню колекцію та змінну для її елементів.
- **on <key1> equals <key2>:** Вказує умову рівності ключів. Увага: використовується слово equals, а не ==!
- **select new { ... }:** Формує фінальний результат проекції об'єднаних полів.
- **Трансляція Roslyn:** Компілятор перетворює цю конструкцію на виклик Enumerable.Join у Method Syntax.

Чому equals, а не ==:

Ключове слово equals підкреслює семантику еквівалентності та використання EqualityComparer<T>.Default.

```
// Query Syntax реляційного з'єднання:
var innerJoin = from s in studentList
                join st in standardList
                on s.StandardID equals st.StandardID
                select new {
                    StudentName = s.StudentName,
                    StandardName = st.StandardName
                };

foreach (var element in innerJoin)
{
    Console.WriteLine($"{element.StudentName}
[{element.StandardName}]");
}

// -> Output:
// -> John [Standard 1]
// -> Moin [Standard 1]
// -> Bill [Standard 2]
// -> Ram [Standard 2]
```

Program.cs

Складені ключі:

Для з'єднання за кількома колонками використовуйте анонімні типи: on new { s.A, s.B } equals new { st.A, st.B }.

Оператор GroupJoin: Ієрархічне з'єднання 1:N у Method Syntax

Групування зв'язаних даних без денормалізації

- **Суть GroupJoin:** З'єднує дві колекції, групуючи всі дочірні елементи під кожним батьківським об'єктом.
- **Колекція у результаті:** Селектор результату приймає батька та IEnumerable<Child> (список студентів для цього класу).
- **Порожні групи:** Якщо для стандарту немає жодного студента, повертається порожня група (без відкидання батька!).
- **Аналог Left Join:** GroupJoin є природною основою для реалізації SQL Left Outer Join у мові C#.

Головна перевага GroupJoin:

Батьківські елементи ніколи не дублюються! Замість 10 рядків з однаковим класом ви отримуєте 1 об'єкт класу зі списком учнів.

```
// Ієрархічне з'єднання 1:N через GroupJoin:
var groupJoin = standardList.GroupJoin(
    studentList,                    // дочірня колекція
    std => std.StandardID,          // ключ батька
    s => s.StandardID,              // ключ дитини
    (std, students) => new {        // результат
        Standard = std.StandardName,
        Students = students
    });

foreach (var item in groupJoin)
{
    Console.WriteLine($"Клас: {item.Standard}");
    foreach (var stud in item.Students)
        Console.WriteLine($"    * {stud.StudentName}");
}

// -> Output:
// -> Клас: Standard 1 -> John, Moin
// -> Клас: Standard 2 -> Bill, Ram, Ron
```

Program.cs

Об'єктна модель:

GroupJoin ідеально лягає на реляційні зв'язки в базах даних: Один факультет -> Багато кафедр -> Багато студентів.

Практичний приклад: Повна ітерація GroupJoin у C#

Аналіз ієрархічного виведення даних

- **Батьківський перебір:** Зовнішній цикл перебирає об'єкти стандартів (Standard 1, Standard 2, Standard 3).
- **Дочірній перебір:** Внутрішній цикл перебирає колекцію студентів item.Students для кожного стандарту.
- **Standard 3 без студентів:** Standard 3 залишається у результаті, але його колекція студентів має Count == 0.
- **Без винятків NullReference:** item.Students завжди є валідним пустим перелічувачем Enumerable.Empty, а не null.

Порівняння з Inner Join:

В Inner Join Standard 3 зник би повністю! У GroupJoin Standard 3 зберігається з порожнім списком.

```
// Виконання та перебір результатів GroupJoin:
var groupJoin = standardList.GroupJoin(
    studentList,
    std => std.StandardID,
    s => s.StandardID,
    (std, students) => new {
        Standard = std.StandardName,
        Students = students
    });

foreach (var item in groupJoin)
{
    Console.WriteLine($"Клас: {item.Standard}");
    foreach (var stud in item.Students)
        Console.WriteLine($"  - {stud.StudentName}");
}

// -> Output:
// -> Клас: Standard 1 -> John, Moin
// -> Клас: Standard 2 -> Bill, Ram, Ron
// -> Клас: Standard 3 (порожньо)
```

Program.cs

Пам'ять процесу:

GroupJoin не створює проміжних списків List<T> — ітерація по студентах відбувається ліниво на вимогу.

GroupJoin у Query Syntax: Конструкція join ... into ...

Ключове слово into для створення груп

- **Синтаксис join into:** Додавання into <groupName> після умови equals перетворює звичайний join на GroupJoin.
- **Змінна studentGroup:** Стає локальною колекцією типу IEnumerable<Student>, що містить усіх учнів поточного класу.
- **Фінальна проекція:** У блоці select можна призначити всю групу у властивість анонімного типу або DTO.
- **Чіткість та лаконічність:** Query Syntax для GroupJoin часто читається легше, ніж довгий 5-параметричний методний ланцюжок.

Ключова відмінність:

Без into: кожен збіг створює окремий плаский рядок. З into: усі збіги упаковуються в єдину локальну групу.

```
// GroupJoin у декларативному Query Syntax:
var groupJoin = from std in standardList
                join s in studentList
                on std.StandardID equals s.StandardID
                into studentGroup
                select new {
                    Students = studentGroup,
                    StandardName = std.StandardName
                };

foreach (var g in groupJoin)
{
    Console.WriteLine($"{g.StandardName}: {g.Students.Count()}
учнів");
}

// -> Output:
// -> Standard 1: 2 учнів
// -> Standard 2: 3 учнів
// -> Standard 3: 0 учнів
```

Program.cs

Ключове правило:

Звичайний join створює плаский декартовий результат; додавання слова into перетворює його на GroupJoin.

Детальний розбір запиту GroupJoin у Query Syntax

Анатомія конструкції join into

- **from std in standardList:** Визначає базове джерело (зовнішня послідовність стандартів).
- **join s in studentList:** Підключає пов'язану колекцію студентів для пошуку збігів.
- **on std.StandardID equals s.StandardID:** Критерій зв'язку за ідентифікатором класу.
- **into studentGroup:** Матеріалізує вибірку пов'язаних студентів у типізовану підколекцію.

Роль into:

studentGroup є локальною змінною діапазону типу IEnumerable<Student>, доступною в наступних пропозиціях запиту.

```
// Повний вираз запиту GroupJoin:
var studentsGroup = from std in standardList
                    join s in studentList
                    on std.StandardID equals s.StandardID
                    into studentGroup
                    select new {
                        Students = studentGroup,
                        StandardName = std.StandardName
                    };

// Дворівневий перебір сформованої ієрархії:
foreach (var group in studentsGroup)
{
    Console.WriteLine(group.StandardName);
    foreach (var s in group.Students)
        Console.WriteLine($"    -> {s.StudentName}");
}

// -> Output: Standard 1 (John, Moin), Standard 2 (Bill, Ram, Ron)
```

Program.cs

Ключове правило:

Звичайний join створює плаский декартовий результат; додавання слова into перетворює його на GroupJoin.

Практичний приклад: Повна ітерація GroupJoin у C#

Аналіз консольного виведення ієрархії

- **Standard 1:** Містить двох студентів: John та Moin.
- **Standard 2:** Містить трьох студентів: Bill, Ram та Ron.
- **Standard 3:** Порожня група, оскільки жоден студент не має StandardID = 3.
- **Трансформація в Left Join:** Виклик DefaultIfEmpty() на групі перетворює її на повноцінний SQL LEFT OUTER JOIN.

Як отримати Left Outer Join:

Якщо після join into викликати from s in studentGroup.DefaultIfEmpty(), ми отримаємо класичний плаский SQL LEFT JOIN!

```
// Реалізація плаского Left Outer Join через DefaultIfEmpty():
var leftOuterJoin = from std in standardList
                    join s in studentList
                    on std.StandardID equals s.StandardID
                    into studentGroup
                    from s in studentGroup.DefaultIfEmpty()
                    select new {
                        Standard = std.StandardName,
                        Student = s != null ? s.StudentName :
"[НЕМАЄ]"
                    };

foreach (var row in leftOuterJoin)
    Console.WriteLine($"{row.Standard}: {row.Student}");

// -> Output:
// -> Standard 1: John
// -> Standard 1: Moin
// -> Standard 2: Bill
// -> Standard 3: [НЕМАЄ]
```

Program.cs

Порада з оптимізації:

GroupJoin набагато ефективніший за ручне знаходження дітей через standardList.Select(std => studentList.Where(...)).

Оператори проекції: Select та SelectMany

Оператор Select (Проекція 1:1)

- **Призначення:** Трансформує кожен елемент послідовності в нову форму.
- **Розмірність:** Кількість елементів зберігається строго однаковою (N -> N).
- **Цільові типи:** Окремі властивості, анонімні типи, класи DTO, Records.

```
// Одновимірна проекція в анонімний тип:
var studentNames = studentList.Select(s => new {
    Name = s.StudentName,
    Age = s.Age
});
// N вхідних елементів -> N вихідних об'єктів
```

Оператор SelectMany (Сплющення 1:N)

- **Призначення:** Розгортає вкладені колекції в єдиний плаский потік (flatMap).
- **Розмірність:** Кількість вихідних елементів може бути більшою або меншою (N -> M).
- **Сценарії:** Студент -> Курси, Клієнт -> Замовлення, Декартів добуток.

```
// Сплющення списку списків у плоский потік:
var allCourses = students.SelectMany(s => s.Courses);

// Було: List<List<Course>>
// Стало: плоский IEnumerable<Course>
```

Функціональна суть:
Select — це математичний Functor Map, а SelectMany — це монадний Bind/FlatMap у теорії категорій.

Практичний приклад Select у Query Syntax C#

Проекція в анонімні типи та властивості

- **Вибірка підмножини полів:** Не обов'язково передавати важкі сутності цілком — можна вибрати лише Name та Age.
- **Анонімні типи new { ... }:** Створюють компактні легковажні об'єкти без ручного опису окремого класу DTO.
- **Перейменування полів:** У select можна перейменовувати властивості: select new { FullName = s.StudentName }.
- **Проекція обчислень:** Можна обчислювати нові значення прямо у виразі: select new { s.StudentName, IsAdult = s.Age >= 18 }.

Економія пам'яті:

Проекція лише потрібних колонок у базах даних генерує точний SQL SELECT Name, Age FROM... замість SELECT *.

```
// 1. Проекція в анонімний тип:
var selectResult = from s in studentList
                   select new {
                       Name = s.StudentName,
                       Age = s.Age,
                       IsAdult = s.Age >= 18
                   };

// 2. Ітерація по проектованих об'єктах:
foreach (var item in selectResult)
{
    Console.WriteLine($"{item.Name} ({item.Age} p.) - Повнолітній:
{item.IsAdult}");
}

// -> Output:
// -> John (13 p.) - Повнолітній: False
// -> Moin (21 p.) - Повнолітній: True
// -> Bill (18 p.) - Повнолітній: True
```

Program.cs

Як працює анонімний тип:

Roslyn генерує незмінний узагальнений клас із перевизначеними Equals() та GetHashCode() за значенням властивостей.

Оператори кванторів: All, Any та Contains

Метод LINQ	Опис	Що робить	Повертає	Приклад	Результат
`All`	Перевіряє, чи всі елементи колекції задовольняють умову.	Повертає true, якщо кожен елемент задовольняє предикат.	bool	`[2, 4, 6].All(x => x % 2 == 0)`	true
`Any`	Перевіряє, чи хоча б один елемент відповідає умові.	Повертає true, як тільки знайдено перший відповідний елемент.	bool	`[1, 2, 3].Any(x => x % 2 == 0)`	true
`Contains`	Перевіряє, чи містить колекція конкретне значення.	Повертає true, якщо заданий елемент знайдено у колекції.	bool	`[1, 2, 3].Contains(2)`	true

```
int[] numbers = { 2, 4, 6, 8, 10 };
bool allEven = numbers.All(n => n % 2 == 0);       // True (всі парні)
bool hasTen  = numbers.Any(n => n == 10);          // True (зупинка на першому 10)
bool hasFive = numbers.Contains(5);                // False (числа 5 немає)
Console.WriteLine($"AllEven: {allEven}, HasTen: {hasTen}, ContainsFive: {hasFive}");
// Output: AllEven: True, HasTen: True, ContainsFive: False
```

QuantifiersDemo.cs

Short-circuiting механіка:
Any зупиняється на першому true, All зупиняється на першому false — вони ніколи не перевіряють зайві елементи даремно.

Оператор All: Квантор загальності (∀)

Перевірка виконання умови для всіх елементів

- **Суть All(predicate):** Оцінює кожен елемент колекції. Повертає true лише тоді, коли предикат виконується для ВСІХ без винятку елементів.
- **Рання зупинка (Short-circuiting):** Як тільки трапляється хоча б ОДИН елемент з false, All негайно завершує роботу і повертає false.
- **Пастка порожньої множини (Vacuous Truth):** Над порожньою колекцією All ЗАВЖДИ повертає true, оскільки контрприкладів не існує!
- **Математичний еквівалент:** Позначає квантор загальності ∀x P(x). Дорівнює запереченню !Any(x => !P(x)).

Практичний приклад:

studentList.All(s => s.Age > 12 && s.Age < 20) поверне false, оскільки Moin (21) та Ram (20) старші за 19 років.

```
// 1. Перевірка: чи всі студенти є підлітками?
bool areAllTeenAgers = studentList.All(
    s => s.Age > 12 && s.Age < 20);
Console.WriteLine($"Усі підлітки: {areAllTeenAgers}");
// Output: Усі підлітки: False (Moin має вік 21)

// 2. Перевірка: чи всі мають вік понад 10 років?
bool allAbove10 = studentList.All(s => s.Age > 10);
Console.WriteLine($"Усі старші 10: {allAbove10}");
// Output: Усі старші 10: True

// 3. Поведінка над порожньою колекцією (Vacuous Truth):
var empty = Enumerable.Empty<Student>();
bool emptyCheck = empty.All(s => s.Age > 100);
Console.WriteLine($"Empty All: {emptyCheck}"); // True!

// -> Roslyn генерує оптимізований цикл з раннім return false
```

Program.cs

Логічний аналог:

All(p) еквівалентний твердженню: !Any(x => !p(x)) (за законом подвійності де Моргана).

Оператор Any: Квантор існування (∃)

Перевірка наявності хоча б одного елемента

- **1. Any() без параметрів:** Перевіряє, чи містить колекція хоча б один елемент за константний час O(1).
- **2. Any(predicate) з умовою:** Перевіряє, чи існує хоча б один елемент, що задовольняє предикат.
- **Миттєве переривання:** Як тільки знайдено перший true, ітерація негайно зупиняється (не читаючи решту).
- **Порожні колекції:** На відміну від All, метод Any() над порожньою колекцією завжди повертає false.

Золоте правило продуктивності:

НІКОЛИ не пишіть if (list.Count() > 0) — це змушує рахувати всі елементи! Завжди пишіть if (list.Any()) за O(1) часу!

```
// 1. Чи є в списку хоча б один підліток?
bool hasTeen = studentList.Any(s => s.Age > 12 && s.Age < 20);
Console.WriteLine($"Є підліток: {hasTeen}");
// Output: Є підліток: True (знайдено John на 1-му кроці!)

// 2. Швидка перевірка на непорожність колекції O(1):
if (studentList.Any())
{
    Console.WriteLine("Список студентів не порожній!");
}

// 3. Пошук неіснуючого елемента:
bool hasSenior = studentList.Any(s => s.Age >= 60);
Console.WriteLine($"Є пенсіонери: {hasSenior}"); // False

// -> Any() негайно повертає true при першому збігу (Short-circuit)
```

Program.cs

Економія ресурсів:

У базах даних Any(predicate) транслюється у надшвидкий SQL: IF EXISTS (SELECT 1 FROM Students WHERE ...).

Оператори агрегації: Скалярні підсумки та математичні операції

Згортка послідовностей у скалярні значення

- **Суть агрегації:** Приймає послідовність елементів і згортає її в одне скалярне число або об'єкт.
- **Негайне виконання (Immediate):** Усі агрегатні оператори виконуються негайно, оскільки вимагають повного перегляду даних.
- **Базовий набір BCL:** Aggregate (універсальний редюсер), Average, Count, LongCount, Max, Min, Sum.
- **Константна пам'ять:** Агрегати не накопичують елементи в пам'яті, обчислюючи результат в одному акумуляторі O(1).

Пастка порожніх списків:

Average, Min та Max викидають InvalidOperationException над порожніми числовими послідовностями!

```
int[] numbers = { 10, 20, 30, 40, 50 };

int totalCount = numbers.Count();     // 5
int totalSum   = numbers.Sum();       // 150
double avg     = numbers.Average();   // 30.0
int minVal     = numbers.Min();       // 10
int maxVal     = numbers.Max();       // 50

// Кастомна агрегація (добуток усіх чисел):
int product = numbers.Aggregate((a, b) => a * b);

Console.WriteLine($"Count: {totalCount}, Sum: {totalSum}, Avg:
{avg}");
Console.WriteLine($"Min: {minVal}, Max: {maxVal}, Product:
{product}");

// -> Output:
// -> Count: 5, Sum: 150, Avg: 30
// -> Min: 10, Max: 50, Product: 12000000
```

Program.cs

Складність за часом:

Усі скалярні агрегати працюють за лінійний час O(N) з константною пам'яттю O(1).

Повна зведена таблиця операторів агрегації LINQ

Метод LINQ	Опис	Що робить	Повертає	Приклад	Результат
`Aggregate`	Кастомна агрегація (множення, конкатенація).	Послідовно накопичує результат через акумулятор.	T / TResult	`[1, 2, 3, 4].Aggregate((a, b) => a * b)`	24
`Average`	Середнє арифметичне числових значень.	Знаходить математичне середнє.	double / decimal	`[1, 2, 3, 4].Average()`	2.5
`Count`	Кількість елементів у колекції.	Рахує всі або ті, що відповідають умові.	int	`[1, 2, 3, 4].Count(x => x > 2)`	2
`LongCount`	Кількість елементів для великих масивів.	Використовується для колекцій розміром > 2 млрд.	long	`[1, 2, 3].LongCount()`	3L
`Max`	Найбільше значення у колекції.	Знаходить максимальний елемент або ключ.	Тип елемента	`[1, 2, 5, 3].Max()`	5
`Min`	Найменше значення у колекції.	Знаходить мінімальний елемент або ключ.	Тип елемента	`[1, 2, 5, 3].Min()`	1
`Sum`	Сума всіх числових елементів.	Додає всі елементи між собою.	int / double / decimal	`[1, 2, 3, 4].Sum()`	10

Безпека типів:
Для безпечного обчислення Average на nullable списках використовуйте list.Cast<double?>().Average() ?? 0.0.

Оператор Aggregate: Функціональний редюсер (Fold / Reduce)

Анатомія накопичувальної операції (Accumulator)

- **Універсальність Aggregate:** Будь-який інший агрегат (Sum, Count, Max) можна виразити через метод Aggregate.
- **Акумуляторний крок:** Функція func(accumulated, nextItem) приймає накопичений проміжний результат та черговий елемент.
- **Три перевантаження:** 1) Без seed (перший елемент є seed); 2) Із явним seed; 3) Із seed та фінальним проектором результату.
- **Імутабельність накопичувача:** Рекомендовано використовувати чисті функції без зміни зовнішнього стану системи.

Схема обчислення (a, b) => a + ", " + b :

Крок 1: 'One' + ', ' + 'Two' -> 'One, Two' | Крок 2: 'One, Two' + ', ' + 'Three'...

```
// 1. Базовий приклад: об'єднання рядків через кому
IList<string> strList = new List<string>() {
    "One", "Two", "Three", "Four", "Five"
};

string commaSeparated =
    strList.Aggregate((s1, s2) => s1 + ", " + s2);
Console.WriteLine(commaSeparated);
// Output: One, Two, Three, Four, Five

// 2. Математичний підрахунок факторіала 5!:
int factorial = Enumerable.Range(1, 5)
    .Aggregate((acc, n) => acc * n);
Console.WriteLine($"5! = {factorial}"); // 120

// 3. Знаходження суми квадратів чисел:
int sumSquares = new[] { 1, 2, 3, 4 }.Aggregate(0, (acc, n) => acc + n
* n);
Console.WriteLine($"Сума квадратів: {sumSquares}"); // 30
```

Program.cs

Важлива деталь:

Якщо послідовність містить лише один елемент, метод Aggregate(func) поверне його без виклику функції.

Практика Aggregate: Конкатенація рядків без зайвих виділень

Покрокове накопичення результату

- **Вхідний список:** strList містить елементи 'One', 'Two', 'Three', 'Four', 'Five'.
- **Крок 1:** s1 = 'One', s2 = 'Two' -> повертає 'One, Two'.
- **Крок 2:** s1 = 'One, Two', s2 = 'Three' -> повертає 'One, Two, Three'.
- **Фінал:** Повертає рядок 'One, Two, Three, Four, Five'.

Порада щодо оптимізації:

Для мільйонів рядків використовуйте StringBuilder у версії Aggregate зі seed, щоб не створювати проміжних рядків у Heap!

```
IList<string> strList = new List<string>() {
    "One", "Two", "Three", "Four", "Five"
};

var commaSeparatedString = strList.Aggregate(
    (s1, s2) => s1 + ", " + s2);

Console.WriteLine(commaSeparatedString);
// Output: One, Two, Three, Four, Five

// Високопродуктивний варіант зі StringBuilder:
var sbResult = strList.Aggregate(
    new StringBuilder(),
    (sb, s) => sb.Append(s).Append(", "),
    sb => sb.ToString().TrimEnd(',', ' '));
```

Зручність C#:

Aggregate дозволяє будувати складні кумулятивні звіти без ручних циклів for та тимчасових змінних.

Перевантаження Aggregate із початковим значенням (Seed)

Агрегація з явним початковим акумулятором

- **Призначення Seed:** Задає стартове значення для акумулятора до початку перегляду елементів колекції.
- **Зміна типу акумулятора:** Дозволяє акумулятору мати тип, відмінний від типу елементів колекції (наприклад, із Student у string).
- **Безпека порожнього списку:** Якщо колекція порожня, Aggregate(seed, func) безпечно поверне seed без винятку!
- **Фінальний селектор:** Третє перевантаження приймає resultSelector для постобробки результату акумулятора.

Сигнатура методу:

Aggregate<TSource, TAccumulate>(seed, (acc, item) => ..., resultSelector)

```
// Студенти для згортки в текстовий рядок:
IList<Student> studentList = new List<Student>() {
    new Student() { StudentID = 1, StudentName = "John", Age = 13 },
    new Student() { StudentID = 2, StudentName = "Moin", Age = 21 },
    new Student() { StudentID = 3, StudentName = "Bill", Age = 18 }
};

// Агрегація зі Seed значенням "Student Names: ":
string studentNames = studentList.Aggregate<Student, string>(
    "Student Names: ",
    (str, s) => str += s.StudentName + ", ");

Console.WriteLine(studentNames);
// -> Output: Student Names: John, Moin, Bill,

// Агрегація у число з початковим балансом 100:
int totalAgeWithBase = studentList.Aggregate(100, (acc, s) => acc +
s.Age);
Console.WriteLine($"Баланс з віком: {totalAgeWithBase}"); // 152
```

Program.cs

Фінальний селектор:

Перевантаження з resultSelector дозволяє на фініші видалити зайву кому: str => str.TrimEnd(',', ' ').

Метод Average: Середнє арифметичне чисел та об'єктів

Average() для числових списків

- **Простий список:** Обчислює середнє безпосередньо для списку чисел intList.
- **Тип результату:** Завжди повертає число з плаваючою крапкою double або decimal.
- **Формула:** Сума всіх чисел, поділена на їхню кількість (150 / 5 = 30.0).

```
IList<int> intList = new List<int>() {
    10, 20, 30, 40, 50
};

var avg = intList.Average();
Console.WriteLine(avg);
// Output: 30
```

Average(selector) для об'єктів

- **Лямбда-селектор:** s => s.Age вказує числову властивість, за якою рахується середнє.
- **Комбінація з Where:** Можна обчислити середній вік лише для певної підгрупи (наприклад, підлітків).
- **Точність:** Запобігає помилкам цілочисельного ділення C#.

```
// Середній вік усіх студентів:
var avgAge = studentList
    .Average(s => s.Age);

// Середній вік підлітків:
var avgTeenAge = studentList
    .Where(s => s.Age < 20)
    .Average(s => s.Age);
Console.WriteLine(avgAge); // 17.4
```

Увага:
Виклик .Average() на порожньому списку кине InvalidOperationException! Завжди перевіряйте наявність елементів.

Метод Count(): Підрахунок загальної кількості та за умовою

Count() без предикату

- **Підрахунок елементів:** Повертає загальну кількість елементів у колекції.
- **Оптимізація ICollection:** Якщо колекція реалізує ICollection, повертає властивість Count без повного перебору!
- **Тип повернення:** Повертає 32-бітне ціле число int.

```
IList<int> intList = new List<int>() {
    10, 21, 30, 45, 50, 87
};

var totalElements = intList.Count();
Console.WriteLine(totalElements);
// Output: 6
```

Count(predicate) з умовою

- **Підрахунок з фільтром:** Рахує лише ті елементи, для яких умова повертає true.
- **Еквівалентність:** intList.Count(p) працює швидше і чистіше, ніж intList.Where(p).Count().
- **Підрахунок парних:** i => i % 2 == 0 рахує числа 10, 30, 50.

```
// Підрахунок парних чисел:
var evenElements = intList
    .Count(i => i % 2 == 0);
Console.WriteLine(evenElements);
// Output: 3

// Студенти старше 18:
var adults = studentList.Count(s => s.Age >= 18);
```

LongCount:
Якщо даних більше 2 мільярдів записів (переповнення int.MaxValue), використовуйте метод LongCount(), що повертає long.

Метод Max: Знаходження найбільшого значення

Пошук максимального елемента або властивості

- **Max() над числами:** Знаходить найбільше число у послідовності intList (серед 10, 21, 30, 45, 50, 87 це 87).
- **Max(selector) над об'єктами:** Повертає максимальне значення вибраної числової властивості (найбільший вік серед студентів).
- **Комбінація з фільтрацією:** Можна шукати максимум серед відфільтрованої підгрупи (найстарший серед підлітків).
- **Новинка MaxBy (.NET 6+):** Замість максимального числа повертає весь об'єкт студента з найбільшим віком!

Відмінність Max від MaxBy (.NET 6+):

Max(s => s.Age) повертає лише ЧИСЛО (21). Новий метод MaxBy(s => s.Age) повертає ЦІЛИЙ ОБ'ЄКТ студента!

```
IList<int> intList = new List<int>() {
    10, 21, 30, 45, 50, 87
};

var largestNumber = intList.Max();
Console.WriteLine($"Max int: {largestNumber}"); // 87

var largestEven = intList.Where(i => i % 2 == 0).Max();
Console.WriteLine($"Max even: {largestEven}"); // 50

// Найстарший студент (повертає число віку):
var oldestAge = studentList.Max(s => s.Age);
Console.WriteLine($"Oldest age: {oldestAge}"); // 21

// Знаходження цілого об'єкта студента через MaxBy (.NET 6+):
var oldestStudent = studentList.MaxBy(s => s.Age);
Console.WriteLine($"Oldest: {oldestStudent.StudentName}"); // Moin
```

Program.cs

Аналогічний Min:

Метод Min() працює абсолютно ідентично, але знаходить найменше значення (наприклад, intList.Min() поверне 10).

Метод Sum: Підсумовування числових значень

Обчислення суми елементів колекції

- **Числова сума:** Додає всі елементи між собою. Підтримує типи int, long, float, double, decimal.
- **Селектор властивості:** studentList.Sum(s => s.Age) обчислює сумарний вік усіх студентів у групі.
- **Безпека порожнього списку:** На відміну від Average/Min/Max, метод Sum() над порожнім списком повертає 0 (без винятку!).
- **Переповнення (Overflow):** Якщо сума int може перевищити 2 млрд, рекомендується приводити числа до long.

Практичне застосування в Enterprise:

Підрахунок загальної вартості кошика замовлень: orders.Sum(o => o.UnitPrice * o.Quantity).

```
IList<int> intList = new List<int>() {
    10, 21, 30, 45, 50, 87
};

var totalSum = intList.Sum();
Console.WriteLine($"Загальна сума: {totalSum}"); // 243

var sumOfEvens = intList.Where(i => i % 2 == 0).Sum();
Console.WriteLine($"Сума парних: {sumOfEvens}"); // 90

// Сума віку підлітків:
var teenAgeSum = studentList
    .Where(s => s.Age < 20)
    .Sum(s => s.Age);
Console.WriteLine($"Сума віку підлітків: {teenAgeSum}"); // 46
```

Program.cs

Переповнення чисел:

Якщо сума може перевищити int.MaxValue, попередньо приведіть числа: numbers.Select(x => (long)x).Sum().

Елементні оператори: ElementAt та ElementAtOrDefault

Доступ до конкретного елемента за індексом

- **Суть ElementAt:** Повертає елемент послідовності за вказаним числовим індексом (нумерація з 0).
- **Оптимізація IList:** Якщо джерело реалізує IList<T>, доступ виконується миттєво O(1). Для потоку — перебір до потрібного кроку.
- **Ризик винятку:** ElementAt викидає ArgumentOutOfRangeException, якщо індекс від'ємний або більший за розмір списку.
- **Безпечний ElementAtOrDefault:** Повертає значення за замовчуванням (0, null), якщо індекс виходить за межі.

Застосування:

intList.ElementAt(1) повертає другий елемент. intList.ElementAtOrDefault(99) безпечно поверне 0.

```
IList<int> intList = new List<int>() { 10, 20, 30, 40, 50 };

// 1. Прямий доступ за індексом:
int second = intList.ElementAt(1);
Console.WriteLine($"Element at index 1: {second}"); // 20

// 2. Безпечний доступ поза межами колекції:
int outOfBounds = intList.ElementAtOrDefault(9);
Console.WriteLine($"Element at index 9: {outOfBounds}"); // 0

// 3. З рядковими колекціями:
IList<string> strList = new List<string>() { "One", "Two", "Three" };
string safeStr = strList.ElementAtOrDefault(10);
Console.WriteLine($"String at index 10: {safeStr ?? "NULL"}"); // NULL

// 4. Доступ до першого та останнього елементів:
int first = intList.ElementAt(0); // 10
int last = intList.ElementAt(intList.Count - 1); // 50
```

Program.cs

Рекомендація:

Для звичайних масивів та List<T> краще використовувати рідний індексатор arr[i]. ElementAt потрібен для довільних IEnumerable<T>.

Повна зведена таблиця елементних операторів LINQ

Метод LINQ	Опис	Що робить	Приклад	Результат
`ElementAt`	Повертає елемент за індексом.	Доступ до конкретного елемента за порядковим номером.	`[10, 20, 30].ElementAt(1)`	20
`ElementAtOrDefault`	Безпечний доступ за індексом.	Повертає default, якщо індекс поза межами.	`[10, 20, 30].ElementAtOrDefault(5)`	0
`First`	Повертає перший елемент або перший за умовою.	Кидає виняток, якщо нічого не знайдено.	`[1, 2, 3].First(x => x > 1)`	2
`FirstOrDefault`	Безпечний пошук першого елемента.	Повертає default, якщо збігів немає.	`[1, 2, 3].FirstOrDefault(x => x > 5)`	0
`Last`	Повертає останній елемент або останній за умовою.	Кидає виняток, якщо колекція порожня.	`[1, 2, 3].Last(x => x < 3)`	2
`LastOrDefault`	Безпечний пошук останнього елемента.	Повертає default, якщо збігів немає.	`[1, 2, 3].LastOrDefault(x => x > 5)`	0
`Single`	Повертає ЄДИНИЙ елемент колекції.	Кидає виняток, якщо елементів 0 або > 1.	`[5].Single()`	5
`SingleOrDefault`	Єдиний елемент або default.	Кидає виняток, якщо знайдено > 1 елемента.	`[5].SingleOrDefault()`	5

Золоте правило суфікса OrDefault:
Усі методи з OrDefault ніколи не викидають виняток при відсутності елементів, а повертають null або 0.

Практичний приклад: ElementAt та ElementAtOrDefault у C#

Числові масиви (int)

- **ElementAt(0):** Повертає перший елемент (10).
- **ElementAt(2):** Повертає третій елемент (30).
- **ElementAtOrDefault(9):** Індекс 9 поза межами — повертає число 0.

```
IList<int> intList = new List<int>() { 10, 20, 30, 40, 50 };

Console.WriteLine(intList.ElementAt(0)); // 10
Console.WriteLine(intList.ElementAt(2)); // 30
Console.WriteLine(intList.ElementAtOrDefault(9)); // 0
```

Рядкові масиви (string)

- **ElementAt(0):** Повертає рядок 'One'.
- **ElementAt(1):** Повертає рядок 'Two'.
- **ElementAtOrDefault(9):** Індекс 9 поза межами — безпечно повертає null (порожньо).

```
IList<string> strList = new List<string>() {
    "One", "Two", "Three", "Four", "Five"
};

Console.WriteLine(strList.ElementAt(0)); // One
Console.WriteLine(strList.ElementAt(1)); // Two
Console.WriteLine(strList.ElementAtOrDefault(9)); // null
```

Пастка винятку:
Виклик intList.ElementAt(9) викине ArgumentOutOfRangeException! Для неперевірених індексів завжди використовуйте ElementAtOrDefault.

Порівняння First та FirstOrDefault

Метод LINQ	Опис	Що робить	Приклад	Результат
`First`	Повертає перший елемент колекції або перший за умовою.	Якщо елементів немає або жоден не відповідає умові — кидає InvalidOperationException.	`[1, 2, 3].First(x => x > 1)`	`2`
`FirstOrDefault`	Повертає перший елемент або перший за умовою.	Якщо елементів немає або жоден не відповідає умові — повертає значення за замовчуванням (default).	`[1, 2, 3].FirstOrDefault(x => x > 5)`	`0 (для int)`

```
IList<int> numbers = new List<int>() { 10, 25, 30, 45, 50 };
int firstEven = numbers.First(n => n % 2 == 0);                    // 10 (знайдено одразу)
int safeMatch = numbers.FirstOrDefault(n => n > 100);              // 0 (безпечно default)
int withDef   = numbers.FirstOrDefault(n => n > 100, -1);          // -1 (кастомний default)
Console.WriteLine($"FirstEven: {firstEven}, SafeMatch: {safeMatch}, WithDef: {withDef}");
// Output: FirstEven: 10, SafeMatch: 0, WithDef: -1
```

FirstVsFirstOrDefault.cs

Кастомне дефолтне значення (.NET 6+):
У сучасному C# можна передавати власне значення за замовчуванням: list.FirstOrDefault(x => x > 5, defaultValue: -1).

Практичний приклад методу First() у C#

Поведінка First() при збігах та на порожніх даних

- **intList.First():** Повертає перший елемент масиву (число 7).
- **intList.First(i => i % 2 == 0):** Повертає перший парний елемент (число 10).
- **emptyList.First():** Викидає виняток InvalidOperationException: Sequence contains no elements!
- **Зупинка конвеєра:** Метод First() завершує сканування негайно при знаходженні першого результату.

Чому First кидає помилку:

First вимагає обов'язкової наявності хоча б одного елемента. Якщо бізнес-логіка допускає відсутність даних, використовуйте FirstOrDefault.

```
IList<int> intList = new List<int>() { 7, 10, 21, 30, 45, 50, 87 };
IList<string> strList = new List<string>() { null, "Two", "Three" };
IList<string> emptyList = new List<string>();

int firstNum = intList.First(); // 7
int firstEven = intList.First(i => i % 2 == 0); // 10
string firstStr = strList.First(); // null

Console.WriteLine($"First: {firstNum}, FirstEven: {firstEven}");

// Спроба виклику на порожньому списку кине помилку:
try {
    emptyList.First();
} catch (InvalidOperationException ex) {
    Console.WriteLine($"Очікуваний виняток: {ex.Message}");
}
// Output: Sequence contains no elements
```

Program.cs

Short-circuiting:

Метод First() завершує роботу на першому ж знайденому елементі і не перебирає залишок колекції.

Практичний приклад FirstOrDefault() у C#

Безпечний пошук без ризику падіння програми

- **intList.FirstOrDefault():** Повертає перший елемент (7).
- **intList.FirstOrDefault(i => i % 2 == 0):** Повертає перший парний елемент (10).
- **emptyList.FirstOrDefault():** Безпечно повертає null (для посилань) або 0 (для чисел) без аварії програми.
- **Перевантаження defaultValue:** У .NET 6+ дозволяє передавати власне значення за замовчуванням, якщо нічого не знайдено.

Рекомендація для веб-сервісів:

У контролерах Web API завжди шукайте сутності через FirstOrDefault(x => x.Id == id) та повертайте NotFound(), якщо результат == null.

```
IList<int> intList = new List<int>() { 7, 10, 21, 30, 45, 50, 87 };
IList<string> emptyList = new List<string>();

int first = intList.FirstOrDefault(); // 7
int firstEven = intList.FirstOrDefault(i => i % 2 == 0); // 10

// Безпечне виконання над порожньою колекцією:
string res = emptyList.FirstOrDefault();
Console.WriteLine($"Результат: {res ?? "[NULL]"}"); // [NULL]

// 2. Пошук з кастомним дефолтним значенням (.NET 6+):
int notFound = intList.FirstOrDefault(i => i > 1000, -1);
Console.WriteLine($"Не знайдено: {notFound}"); // -1

// 3. Безпечний пошук сутності:
Student std = studentList.FirstOrDefault(s => s.StudentID == 99);
```

Program.cs

Порада з безпеки:

Якщо default(T) для int є 0, а 0 є валідним бізнес-значенням, передавайте кастомний default: list.FirstOrDefault(p, -1).

Оператори Last та LastOrDefault

Метод LINQ	Опис	Що робить	Приклад	Результат
`Last`	Повертає останній елемент колекції або останній за умовою.	Якщо немає підходящих елементів — кидає виняток.	`[1, 2, 3, 4].Last(x => x < 4)`	`3`
`LastOrDefault`	Повертає останній елемент або останній за умовою.	Якщо немає підходящих елементів — повертає default.	`[1, 2, 3, 4].LastOrDefault(x => x > 5)`	`0 (для int)`

```
IList<int> numbers = new List<int>() { 10, 21, 30, 45, 50, 87 };
int lastNum  = numbers.Last();                                  // 87 (останнє число)
int lastEven = numbers.Last(n => n % 2 == 0);                   // 50 (останнє парне)
int notFound = numbers.LastOrDefault(n => n > 100);             // 0 (безпечно default)
Console.WriteLine($"Last: {lastNum}, LastEven: {lastEven}, NotFound: {notFound}");
// Output: Last: 87, LastEven: 50, NotFound: 0
```

LastVsLastOrDefault.cs

Продуктивність Last():
Якщо колекція є потоком (не масивом), Last змушений прочитати ВСЮ колекцію до самого кінця, щоб знайти останній елемент.

Практичний приклад методу Last() у C#

Пошук останнього елемента у списку

- **intList.Last():** Повертає останній елемент масиву (число 87).
- **intList.Last(i => i % 2 == 0):** Повертає останній парний елемент масиву (число 50).
- **strList.Last():** Повертає останній рядок списку ('Five').
- **emptyList.Last():** Викидає виняток InvalidOperationException на порожній колекції.

Оптимізація IList:

Для List<T> або T[] метод Last() миттєво бере елемент за індексом list[list.Count - 1] за O(1) часу.

```
IList<int> intList = new List<int>() { 7, 10, 21, 30, 45, 50, 87 };
IList<string> strList = new List<string>() { null, "Two", "Three",
"Four", "Five" };

int lastInt = intList.Last(); // 87
int lastEven = intList.Last(i => i % 2 == 0); // 50
string lastStr = strList.Last(); // Five

Console.WriteLine($"Last: {lastInt}, LastEven: {lastEven}, LastStr:
{lastStr}");

// Спроба викликати Last на порожньому списку кине виняток:
var empty = new List<int>();
try {
    empty.Last();
} catch (InvalidOperationException ex) {
    Console.WriteLine($"Помилка: {ex.Message}");
}
```

Program.cs

Важлива відмінність:

First() може завершитися на 1-му кроці, але Last(predicate) завжди змушений сканувати колекцію до кінця.

Практичний приклад LastOrDefault() у C#

Безпечний пошук останнього елемента

- **intList.LastOrDefault():** Повертає останній елемент (87).
- **intList.LastOrDefault(i => i % 2 == 0):** Повертає останній парний елемент (50).
- **emptyList.LastOrDefault():** Безпечно повертає 0 або null без винятку.
- **Пошук транзакцій:** Ідеально для знаходження останньої виконаної дії або останнього запису в журналі аудиту.

Сфера застосування:

Пошук останньої виконаної транзакції користувача або останнього доданого запису в лог-файл.

```
IList<int> intList = new List<int>() { 7, 10, 21, 30, 45, 50, 87 };
IList<string> emptyList = new List<string>();

int last = intList.LastOrDefault(); // 87
int lastEven = intList.LastOrDefault(i => i % 2 == 0); // 50

// Безпечне виконання над порожнім списком:
var lastEmpty = emptyList.LastOrDefault();
Console.WriteLine(lastEmpty == null ? "NULL" : lastEmpty); // NULL

// 2. Пошук з умовою без збігів:
int noMatch = intList.LastOrDefault(i => i > 500);
Console.WriteLine($"No match: {noMatch}"); // 0

// 3. Кастомний default у .NET 6+:
int defMatch = intList.LastOrDefault(i => i > 500, -1); // -1
```

Program.cs

Порада:

Якщо потрібен останній елемент відсортованої бази даних, ефективніше зробити .OrderByDescending().FirstOrDefault().

Поведінка Last проти LastOrDefault при відсутності збігів

Коли умова фільтра не знаходить жодного елемента

- **intList.Last(i => i > 250):** Жодне число в масиві не перевищує 250.
- **Реакція Last():** Викидає InvalidOperationException: Sequence contains no matching element!
- **Реакція LastOrDefault():** Безпечно повертає значення default (0 або null) без аварії.
- **Діагностика продакшену:** Помилка 'Sequence contains no matching element' є маркером використання Last замість LastOrDefault.

Золоте правило стабільності:

Якщо відповідність предикату не гарантована на 100%, завжди використовуйте LastOrDefault() з наступною перевіркою результату.

```
IList<int> intList = new List<int>() { 7, 10, 21, 30, 45, 50, 87 };
IList<string> strList = new List<string>() { null, "Two", "Three",
"Four", "Five" };

// 1. Last з умовою без збігів КИДАЄ ВИЙНЯТОК:
// intList.Last(i => i > 250);
// -> Run-time exception: Sequence contains no matching element

// 2. LastOrDefault з умовою без збігів БЕЗПЕЧНИЙ:
int notFound = intList.LastOrDefault(i => i > 250);
Console.WriteLine($"Не знайдено: {notFound}"); // 0

var strFound = strList.LastOrDefault(s => s != null &&
s.Contains('T'));
Console.WriteLine($"Знайдено рядок: {strFound}"); // Three

// 3. З кастомним default значенням (.NET 6+):
int safeCustom = intList.LastOrDefault(i => i > 500, defaultValue: -
1);
Console.WriteLine($"Кастомний результат: {safeCustom}"); // -1
```

Program.cs

Діагностика помилок:

Помилка 'Sequence contains no matching element' є типовим маркером некоректного використання First()/Last() замість OrDefault.

Оператори Single та SingleOrDefault: Перевірка унікальності

Метод LINQ	Опис	Що робить	Приклад	Результат
`Single`	Повертає ЄДИНИЙ елемент колекції або єдиний за умовою.	Якщо елементів немає або їх більше одного — кидає InvalidOperationException.	`[5].Single()`	5
`SingleOrDefault`	Повертає єдиний елемент або значення default.	Якщо елементів немає — повертає default. Якщо знайдено БІЛЬШЕ одного — кидає InvalidOperationException!	`[5].SingleOrDefault()`	5

```
IList<int> singleItem = new List<int>() { 42 };
IList<int> multiples  = new List<int>() { 10, 20, 30 };
int exactlyOne = singleItem.Single();                    // 42 (успіх: рівно 1 елемент)
int safeEmpty  = new List<int>().SingleOrDefault();      // 0 (безпечно: 0 елементів)
// multiples.Single(); // КРИТИЧНИЙ ВИЙНЯТОК: знайдено більше 1 елемента!
Console.WriteLine($"ExactlyOne: {exactlyOne}, SafeEmpty: {safeEmpty}");
// Output: ExactlyOne: 42, SafeEmpty: 0
```

SingleVsSingleOrDefault.cs

Критична пастка SingleOrDefault:
SingleOrDefault повертає default ТІЛЬКИ коли елементів 0. Якщо знайдено 2 або більше — він ВСЕ ОДНО кидає виняток!

Практичний приклад Single та SingleOrDefault у C#

Жорсткий контроль унікальності бізнес-даних

- **Колекція з 1 елементом:** oneElementList.Single() успішно повертає число 7.
- **Колекція з багатьма елементами:** intList.Single() кидає виняток, оскільки містить більше одного числа!
- **Фільтр з 1 результатом:** intList.Single(i => i < 10) успішно повертає 7 (єдине число менше 10).
- **Порожній список:** emptyList.SingleOrDefault() повертає 0 без помилок.

Коли використовувати Single:

Виключно тоді, коли поява другого дубліката є критичною аварією цілісності бази даних (наприклад, пошук за унікальним GUID/User ID).

```
IList<int> oneElementList = new List<int>() { 7 };
IList<int> intList = new List<int>() { 7, 10, 21, 30, 45, 50, 87 };
IList<int> emptyList = new List<int>();

// 1. Успішні виклики унікальних значень:
int singleVal = oneElementList.Single(); // 7
int emptyVal  = emptyList.SingleOrDefault(); // 0
int singleFiltered = intList.Single(i => i < 10); // 7

Console.WriteLine($"Single: {singleVal}, Default: {emptyVal},
Filtered: {singleFiltered}");

// 2. Винятки через дублікати (знайдено більше 1):
// intList.Single(); // InvalidOperationException!
// intList.SingleOrDefault(); // InvalidOperationException!
// emptyList.Single(); // InvalidOperationException!
```

Program.cs

Продуктивність у БД:

SingleOrDefault змушує EF Core виконувати SELECT TOP(2), щоб перевірити відсутність дубліката, тоді як FirstOrDefault робить TOP(1).

Оператор рівності послідовностей: SequenceEqual

Перевірка ідентичності двох колекцій

- **Єдиний оператор рівності:** У LINQ існує тільки один оператор рівності — SequenceEqual.
- **Три критерії збігу:** Метод перевіряє: 1) однакову кількість елементів, 2) попарну рівність значень, 3) абсолютно однаковий порядок слідування.
- **Порівняння примітивів:** Для простих типів (string, int) використовується стандартний компаратор EqualityComparer<T>.Default.
- **Порядок має значення:** Навіть якщо колекції містять абсолютно ті самі елементи, але в іншому порядку — повертається false.

Поведінка методу:

Обхід обох колекцій відбувається синхронно через два ітератори MoveNext() паралельно.

```
// Приклад 1: Однаковий вміст і однаковий порядок
IList<string> strList1 = new List<string>() {
    "One", "Two", "Three", "Four", "Three"
};
IList<string> strList2 = new List<string>() {
    "One", "Two", "Three", "Four", "Three"
};
bool isEqual1 = strList1.SequenceEqual(strList2);
Console.WriteLine(isEqual1); // true

// Приклад 2: Однаковий вміст, але різний порядок
IList<string> strList3 = new List<string>() { "One", "Two", "Three" };
IList<string> strList4 = new List<string>() { "Two", "One", "Three" };
bool isEqual2 = strList3.SequenceEqual(strList4);
Console.WriteLine(isEqual2); // false!

-> Output: True
-> Output: False
Program.cs
```

Важливе правило:

SequenceEqual повертає true виключно за умови повного попарного збігу елементів та їхніх індексів у послідовності.

Порівняння об'єктів Student за допомогою SequenceEqual та IEqualityComparer

Посилальні типи та кастомний компаратор

- **Проблема посилань:** За замовчуванням для класів C# порівнює посилання в купі (ReferenceEquals). Два екземпляри з однаковими полями вважаються різними!
- **Кастомний компаратор:** Щоб порівнювати екземпляри Student за бізнес-полями (StudentID, StudentName), передається IEqualityComparer<Student>.
- **Реалізація методів:** Компаратор повинен реалізувати Equals(Student x, Student y) та узгоджений GetHashCode(Student obj).
- **Вимога узгодженості:** Якщо Equals повертає true для двох об'єктів, їхні GetHashCode обов'язково повинні збігатися для коректного хешування.

Результат порівняння:

За наявності StudentComparer метод повертає true, оскільки всі відповідні студенти мають однаковий ID та ім'я.

```
IList<Student> studentList1 = new List<Student>() {
    new Student() { StudentID = 1, StudentName = "John", Age = 18 },
    new Student() { StudentID = 2, StudentName = "Steve", Age = 15 },
    new Student() { StudentID = 3, StudentName = "Bill", Age = 25 }
};

IList<Student> studentList2 = new List<Student>() {
    new Student() { StudentID = 1, StudentName = "John", Age = 18 },
    new Student() { StudentID = 2, StudentName = "Steve", Age = 15 },
    new Student() { StudentID = 3, StudentName = "Bill", Age = 25 }
};

// Без компаратора поверне false (різні екземпляри в пам'яті)
// З кастомним компаратором порівнюються поля сутності:
bool isEqual = studentList1.SequenceEqual(
    studentList2, new StudentComparer());

Console.WriteLine(isEqual); // True
-> Output: True
```

StudentComparer.cs

Угода про хешування:

Метод GetHashCode компаратора зобов'язаний повертати однаковий код для об'єктів, які метод Equals вважає рівними.

Оператор Concat: Послідовне злиття двох списків рядків

Послідовне приєднання колекцій

- **Принцип Concat:** Метод Concat приєднує другу послідовність у кінець першої, створюючи єдиний потік даних.
- **Збереження дублікатів:** На відміну від Union, оператор Concat НЕ усуває повторювані елементи і не змінює порядок.
- **Швидкодія O(N + M):** Метод не будує хеш-таблиць у пам'яті, тому працює надзвичайно швидко з мінімальними накладними витратами.
- **Потокова передача:** Concat транслюється у два послідовних цикли foreach, не виділяючи проміжних буферних масивів у пам'яті.

Результат виклику:

Створюється послідовність: One, Two, Three, Five, Six. Елементи обходяться ліниво один за одним.

```
IList<string> collection1 = new List<string>() {
    "One", "Two", "Three"
};

IList<string> collection2 = new List<string>() {
    "Five", "Six"
};

// Послідовне злиття двох списків
var collection3 = collection1.Concat(collection2);

foreach (string str in collection3)
    Console.WriteLine(str);

-> Output:
-> One
-> Two
-> Three
-> Five
-> Six
```

Program.cs

Відмінність Concat від Union:

Concat просто конкатенує послідовності без перевірки на унікальність. Якщо потрібна множина без повторів — використовуйте Union.

Оператор Concat: Злиття числових послідовностей int

Конкатенація примітивних значень

- **Об'єднання чисел:** Демонстрація злиття двох числових послідовностей: [1, 2, 3] та [4, 5, 6].
- **Лінивий конвеєр:** Використовує механізм yield return: спочатку повністю читається перша колекція, а потім — друга.
- **Ланцюговий виклик:** Можна об'єднувати декілька джерел: col1.Concat(col2).Concat(col3).
- **Типізація результату:** Повертає IEnumerable<int>, зберігаючи всі повтори та вихідні позиції елементів кожного з джерел.

Економія пам'яті:

Поки не викликано .ToList() або .ToArray(), новий масив не виділяється в пам'яті.

```
IList<int> collection1 = new List<int>() { 1, 2, 3 };
IList<int> collection2 = new List<int>() { 4, 5, 6 };

// Об'єднання двох числових послідовностей
var collection3 = collection1.Concat(collection2);

foreach (int i in collection3)
    Console.WriteLine(i);

// Ланцюгове об'єднання з третьою колекцією:
var collection4 = collection3.Concat(new List<int>() { 7, 8 });
Console.WriteLine("Всього елементів: " + collection4.Count());

-> Output:
-> 1, 2, 3, 4, 5, 6
-> Всього елементів: 8
```

Program.cs

Порядок слідування:

Гарантується строгий порядок: усі елементи першої колекції передують елементам другої колекції.

Оператори роботи з множинами: Distinct, Except, Intersect, Union

Математичні операції над множинами в C#

- **Чотири оператори множин:** LINQ реалізує класичні теоретико-множинні операції: видалення дублікатів, різницю, перетин та об'єднання.
- **Усунення повторень:** Усі оператори над множинами повертають вибірки, де кожен унікальний елемент представлений лише один раз.
- **Внутрішня реалізація:** Для швидкого пошуку дублікатів за O(1) LINQ використовує внутрішній хеш-набір Set<T>.
- **Множинні перевантаження:** Кожен метод має перевантаження з IEqualityComparer<T> для порівняння складних сутностей за полями.

Підтримка компараторів:

Кожен оператор має перевантаження з параметром IEqualityComparer<T> для налаштування порівняння складних класів.

Призначення операторів множин

- **Distinct:** Повертає лише унікальні значення з однієї вихідної послідовності.
- **Except:** Повертає різницю двох множин: елементи першої колекції, яких немає в другій.
- **Intersect:** Повертає перетин двох множин: спільні елементи, які є в обох колекціях одночасно.
- **Union:** Повертає математичне об'єднання двох колекцій без дублювання спільних елементів.

Deferred Execution:

Усі оператори множин обчислюються відкладено і не матеріалізують результат до початку циклу або .ToList().

Теорія множин:

Оператори множин LINQ повністю повторюють операції над множинами в реляційних базах даних (DISTINCT, EXCEPT, INTERSECT, UNION).

Зведена порівняльна таблиця операторів над множинами

Метод LINQ	Опис / Призначення	Що робить	Приклад у C#	Результат
`Distinct`	Повертає унікальні значення з колекції.	Видаляє всі дублікати з поточної послідовності.	`[1, 2, 2, 3].Distinct()`	[1, 2, 3]
`Except`	Повертає різницю між двома колекціями.	Елементи першої колекції, яких немає в другій.	`[1, 2, 3].Except([2, 3, 4])`	[1]
`Intersect`	Повертає спільні елементи двох колекцій.	Вибирає елементи, які є одночасно в обох списках.	`[1, 2, 3].Intersect([2, 3, 4])`	[2, 3]
`Union`	Об'єднує дві колекції без повторень.	Всі елементи обох колекцій з усуненням дублікатів.	`[1, 2].Union([2, 3])`	[1, 2, 3]

```
// Демонстрація операцій над множинами в C# (Set Operators):
var colA = new int[] { 1, 2, 2, 3 };
var colB = new int[] { 2, 3, 4 };
var distinct  = colA.Distinct();          // -> [1, 2, 3] (дублікати числа 2 видалено)
var except    = colA.Except(colB);        // -> [1] (елементи colA, яких немає в colB)
var intersect = colA.Intersect(colB);     // -> [2, 3] (спільні елементи для обох колекцій)
var union     = colA.Union(colB);         // -> [1, 2, 3, 4] (об'єднання без дублювання)
Console.WriteLine($"Distinct:  {string.Join(", ", distinct)}");
Console.WriteLine($"Except:    {string.Join(", ", except)}");
Console.WriteLine($"Intersect: {string.Join(", ", intersect)}");
Console.WriteLine($"Union:     {string.Join(", ", union)}");
```

SetOperatorsDemo.cs

Резюме поведінки:

Усі 4 методи повертають IEnumerable<T> та гарантують абсолютну унікальність кожного значення у вихідній послідовності.

Візуалізація операцій на прикладі тестових множин

Distinct та Except

- **Вхідна колекція 1:** Col1 = { "One", "Two", "Three", "Two" }
- **Distinct(Col1):** Видаляє другий "Two" -> { "One", "Two", "Three" }
- **Except(Col1, Col2):** Де Col2 = { "Two", "Four" } -> повертає { "One", "Three" } (елементи з Col1, яких нема в Col2)

```
// 1. Distinct - усунення повторів:
var dist = col1.Distinct();
// -> One, Two, Three

// 2. Except - різниця множин:
var exc = col1.Except(col2);
// -> One, Three
```

Intersect та Union

- **Вхідна колекція 2:** Col2 = { "Two", "Four", "Five" }
- **Intersect(Col1, Col2):** Спільні елементи для обох списків -> { "Two" }
- **Union(Col1, Col2):** Повне об'єднання всіх унікальних -> { "One", "Two", "Three", "Four", "Five" }

```
// 3. Intersect - перетин множин:
var inter = col1.Intersect(col2);
// -> Two

// 4. Union - об'єднання без дублікатів:
var un = col1.Union(col2);
// -> One, Two, Three, Four, Five
```

Комутативність:
Intersect та Union є комутативними за складом (A та B дають однакові множини). Except НЕ є комутативним: A \ B != B \ A.

Оператор Distinct: Очищення рядків та чисел від дублікатів

Фільтрація унікальних значень

- **Призначення:** Distinct повертає кожне неповторюване значення з колекції рівно один раз.
- **Збереження порядку:** Порядок першої появи кожного елемента гарантовано зберігається у вихідному результаті.
- **Чутливість до регістру:** Для рядків за замовчуванням 'Three' і 'three' є різними значеннями. Для нечутливого порівняння передають StringComparer.OrdinalIgnoreCase.
- **Потокова фільтрація:** Distinct віддає унікальні елементи на льоту через yield return, як тільки зустрічає нове неповторюване значення.

Результат для списків:

Рядки: One, Two, Three (повтори відкинуто). Числа: 1, 2, 3, 4, 5 (повтори чисел 2, 4, 3 відкинуто).

```
IList<string> strList = new List<string>() {
    "One", "Two", "Three", "Two", "Three"
};
IList<int> intList = new List<int>() { 1, 2, 3, 2, 4, 4, 3, 5 };

var distinctList1 = strList.Distinct();
foreach (var str in distinctList1)
    Console.WriteLine(str);

var distinctList2 = intList.Distinct();
foreach (var i in distinctList2)
    Console.WriteLine(i);

-> Output рядків: One, Two, Three
-> Output чисел: 1, 2, 3, 4, 5
```

Program.cs

Складність операції:

Distinct працює з часовою складністю O(N) завдяки використанню внутрішньої хеш-таблиці.

Оператор Distinct для об'єктів Student з StudentComparer

Усунення дублікатів складних об'єктів

- **Проблема посилань:** У списку є 4 студенти з ім'ям 'Bill', ID = 3, Age = 25. Без компаратора Distinct поверне всі 4 об'єкти, бо це різні адреси в купі!
- **Передача StudentComparer:** Передаючи екземпляр IEqualityComparer<Student>, ми змушуємо Distinct порівнювати StudentID та StudentName.
- **Сучасна альтернатива:** У C# 9+ використання record struct або record class забезпечує автопорівняння за значеннями без окремих компараторів.
- **Хешування об'єктів:** Distinct спочатку викликає GetHashCode, а лише при колізії хешів — дорожчий метод Equals.

Результат фільтрації:

Повертаються лише 4 унікальні студенти: John, Steve, Bill (один раз) та Ron.

```
IList<Student> studentList = new List<Student>() {
    new Student() { StudentID = 1, StudentName = "John", Age = 18 },
    new Student() { StudentID = 2, StudentName = "Steve", Age = 15 },
    new Student() { StudentID = 3, StudentName = "Bill", Age = 25 },
    new Student() { StudentID = 3, StudentName = "Bill", Age = 25 },
    new Student() { StudentID = 3, StudentName = "Bill", Age = 25 },
    new Student() { StudentID = 3, StudentName = "Bill", Age = 25 },
    new Student() { StudentID = 5, StudentName = "Ron", Age = 19 }
};

var distinctStudents = studentList.Distinct(
    new StudentComparer());

foreach (Student std in distinctStudents)
    Console.WriteLine(std.StudentName);

-> Output: John, Steve, Bill, Ron
```

StudentComparer.cs

Зверніть увагу:

У вихідний результат потрапляє перший зустрінутий екземпляр з групи еквівалентних об'єктів.

Оператор Except: Різниця між двома колекціями рядків

Обчислення різниці послідовностей (A \ B)

- **Принцип Except:** Повертає елементи першої колекції, які відсутні в другій колекції (параметрі).
- **Видалення збігів:** Елементи 'Four' та 'Five' є в обох списках, тому вони повністю вилучаються з результату.
- **Елементи другої колекції:** Елементи 'Six', 'Seven', 'Eight' із strList2 взагалі не потрапляють у результат, бо їх не було в strList1.
- **Одноразове сканування:** Друга колекція зчитується в хеш-сет Set<T>, після чого перший список фільтрується за лінійний час O(N + M).

Результат вибірки:

Повертаються тільки: One, Two, Three. Усі спільні та сторонні значення відсіяні.

```
IList<string> strList1 = new List<string>() {
    "One", "Two", "Three", "Four", "Five"
};

IList<string> strList2 = new List<string>() {
    "Four", "Five", "Six", "Seven", "Eight"
};

// Елементи strList1, яких немає в strList2
var result = strList1.Except(strList2);

foreach (string str in result)
    Console.WriteLine(str);

-> Output:
-> One
-> Two
-> Three
```

Program.cs

Некомутативність:

strList2.Except(strList1) повернув би зовсім інший результат: Six, Seven, Eight.

Оператор Except для об'єктів Student з StudentComparer

Вилучення підмножини бізнес-об'єктів

- **Вихідний список 1:** Містить студентів: John (1), Steve (2), Bill (3), Ron (5).
- **Список вилучення 2:** Містить студентів: Bill (3), Ron (5).
- **Порівняння компаратором:** Завдяки StudentComparer метод Except знаходить відповідність за ID та іменем і коректно віднімає студентів.
- **Ідентифікація сутностей:** StudentComparer зіставляє ID та ім'я, забезпечуючи коректне вилучення студентів із бази.

Підсумок різниці:

У результуючій колекції залишаються тільки John та Steve.

```
IList<Student> studentList1 = new List<Student>() {
    new Student() { StudentID = 1, StudentName = "John", Age = 18 },
    new Student() { StudentID = 2, StudentName = "Steve", Age = 15 },
    new Student() { StudentID = 3, StudentName = "Bill", Age = 25 },
    new Student() { StudentID = 5, StudentName = "Ron", Age = 19 }
};
IList<Student> studentList2 = new List<Student>() {
    new Student() { StudentID = 3, StudentName = "Bill", Age = 25 },
    new Student() { StudentID = 5, StudentName = "Ron", Age = 19 }
};

var resultedCol = studentList1.Except(
    studentList2, new StudentComparer());

foreach (Student std in resultedCol)
    Console.WriteLine(std.StudentName);

-> Output: John, Steve
```

StudentComparer.cs

Практичне застосування:

Швидкий пошук неактивних користувачів, боржників або студентів, які ще не записані на обрану дисципліну.

Оператор Intersect: Перетин двох списків рядків

Пошук спільних елементів (A ∩ B)

- **Принцип Intersect:** Метод повертає нову колекцію, що містить тільки ті елементи, які присутні в обох джерелах одночасно.
- **Унікальність результату:** У вихідній послідовності гарантовано відсутні дублікати, навіть якщо в джерелах спільний елемент повторювався.
- **Порядок елементів:** Порядок елементів визначається порядком їх появи в першій колекції (strList1).
- **Хеш-перетин:** LINQ створює хеш-набір з другої колекції і повертає елементи першої, що знайдені в ньому, видаляючи їх для уникнення дублікатів.

Результат перетину:

Спільними для двох списків є лише 'Four' та 'Five'.

```
IList<string> strList1 = new List<string>() {
    "One", "Two", "Three", "Four", "Five"
};

IList<string> strList2 = new List<string>() {
    "Four", "Five", "Six", "Seven", "Eight"
};

// Пошук спільних елементів
var result = strList1.Intersect(strList2);

foreach (string str in result)
    Console.WriteLine(str);

-> Output:
-> Four
-> Five
```

Program.cs

Бізнес-застосування:

Перевірка перетину прав доступу користувача, знаходження спільних спільників або спільних тегів товарів.

Оператор Intersect для об'єктів Student з StudentComparer

Знаходження спільних студентів у двох групах

- **Колекція 1:** John (1), Steve (2), Bill (3), Ron (5).
- **Колекція 2:** Bill (3), Ron (5).
- **Перетин списків:** Intersect порівнює сутності через StudentComparer і повертає спільних студентів.
- **Спільний реєстр:** Знаходить тих самих студентів у двох різних академічних групах або списках курсів.

Результат вибірки:

Повертаються об'єкти Bill та Ron, які присутні в обох колекціях одночасно.

```
IList<Student> studentList1 = new List<Student>() {
    new Student() { StudentID = 1, StudentName = "John", Age = 18 },
    new Student() { StudentID = 2, StudentName = "Steve", Age = 15 },
    new Student() { StudentID = 3, StudentName = "Bill", Age = 25 },
    new Student() { StudentID = 5, StudentName = "Ron", Age = 19 }
};
IList<Student> studentList2 = new List<Student>() {
    new Student() { StudentID = 3, StudentName = "Bill", Age = 25 },
    new Student() { StudentID = 5, StudentName = "Ron", Age = 19 }
};

var resultedCol = studentList1.Intersect(
    studentList2, new StudentComparer());

foreach (Student std in resultedCol)
    Console.WriteLine(std.StudentName);

-> Output: Bill, Ron
```

StudentComparer.cs

Джерело екземплярів:

Об'єкти у вихідній послідовності повертаються з першої колекції (studentList1).

Оператор Union: Математичне об'єднання списків рядків

Об'єднання множин без дублювання (A ∪ B)

- **Принцип Union:** Об'єднує дві послідовності та повертає всі унікальні елементи обох списків.
- **Видалення дублікатів:** Значення 'Two' є в обох списках, але в результаті з'являється лише один раз.
- **Чутливість до регістру:** 'three' та 'THREE' розглядаються як різні рядки і обидва потрапляють у вибірку.
- **Ліниве злиття:** Union послідовно читає обидва джерела, видаючи кожен новий неповторюваний елемент через yield return.

Результат вибірки:

One, Two, three, THREE, Four, Five. Забезпечено повну унікальність значень.

```
IList<string> strList1 = new List<string>() {
    "One", "Two", "three"
};

IList<string> strList2 = new List<string>() {
    "Two", "THREE", "Four", "Five"
};

// Об'єднання двох множин з усуненням дублікатів
var result = strList1.Union(strList2);

foreach (string str in result)
    Console.WriteLine(str);

-> Output:
-> One, Two, three, THREE, Four, Five
Program.cs
```

Порівняння з Concat:

Якби замість Union було викликано Concat, 'Two' повторилося б двічі.

Оператор Union для об'єктів Student з StudentComparer

Об'єднання реєстрів студентів без дублювання

- **Колекція 1:** John (1), Steve (2), Bill (3), Ron (5).
- **Колекція 2:** Bill (3), Ron (5).
- **Усунення повторів:** Оскільки Bill та Ron вже є в studentList1, метод Union завдяки StudentComparer не додає їх повторно.
- **Консолідація баз даних:** Ідеально підходить для дедуплікації даних при синхронізації кількох локальних реєстрів в один.

Результат об'єднання:

John, Steve, Bill, Ron — єдиний консолідований список унікальних студентів.

```
IList<Student> studentList1 = new List<Student>() {
    new Student() { StudentID = 1, StudentName = "John", Age = 18 },
    new Student() { StudentID = 2, StudentName = "Steve", Age = 15 },
    new Student() { StudentID = 3, StudentName = "Bill", Age = 25 },
    new Student() { StudentID = 5, StudentName = "Ron", Age = 19 }
};
IList<Student> studentList2 = new List<Student>() {
    new Student() { StudentID = 3, StudentName = "Bill", Age = 25 },
    new Student() { StudentID = 5, StudentName = "Ron", Age = 19 }
};

var resultedCol = studentList1.Union(
    studentList2, new StudentComparer());

foreach (Student std in resultedCol)
    Console.WriteLine(std.StudentName);

-> Output: John, Steve, Bill, Ron
```

StudentComparer.cs

Підсумок блоку Set Operators:

Усі 4 оператори множин (Distinct, Except, Intersect, Union) вимагають компаратора при роботі з користувацькими класами.

Оператори розбиття послідовностей: Skip, SkipWhile, Take, TakeWhile

Метод LINQ	Опис / Призначення	Що робить	Приклад у C#	Результат
`Skip`	Пропускає задану кількість елементів з початку послідовності.	Пропускає перші N елементів і повертає всі наступні.	`[1, 2, 3, 4].Skip(2)`	[3, 4]
`SkipWhile`	Пропускає елементи з початку, поки умова true.	Як тільки умова стає false — припиняє пропуск і повертає решту послідовності.	`[1, 2, 3, 4].SkipWhile(x => x < 3)`	[3, 4]
`Take`	Бере задану кількість елементів з початку послідовності.	Повертає тільки перші N елементів і завершує вибірку.	`[1, 2, 3, 4].Take(2)`	[1, 2]
`TakeWhile`	Бере елементи з початку, поки умова true.	Як тільки умова стає false — негайно зупиняє подальшу вибірку.	`[1, 2, 3, 4].TakeWhile(x => x < 3)`	[1, 2]

```
// Канонічний патерн сторінкової пагінації (Pagination Pattern):
int pageNumber = 2, pageSize = 3;
var items = Enumerable.Range(1, 10); // послідовність чисел 1..10
var page = items.Skip((pageNumber - 1) * pageSize).Take(pageSize);
Console.WriteLine($"Сторінка {pageNumber}: {string.Join(", ", page)}"); // 4, 5, 6

// Умовне позиційне секціонування (TakeWhile / SkipWhile):
var whileTake = items.TakeWhile(x => x < 4); // [1, 2, 3] (до першого false)
var whileSkip = items.SkipWhile(x => x < 7); // [7, 8, 9, 10] (пропуск до першого false)
Console.WriteLine($"TakeWhile: {string.Join(", ", whileTake)}");
Console.WriteLine($"SkipWhile: {string.Join(", ", whileSkip)}");
```

PaginationPattern.cs

Патерн Pagination:

Спільне використання: source.Skip((pageNumber - 1) * pageSize).Take(pageSize) реалізує сторінкову розбивку в UI та API.

Оператор Skip: Пропуск фіксованої кількості початкових елементів

Пропуск перших N елементів

- **Принцип роботи:** Skip(count) ігнорує перші count елементів у послідовності та повертає всі інші.
- **Якщо count <= 0:** Метод повертає всю вихідну послідовність без змін.
- **Якщо count >= розміру:** Повертається порожня колекція (виняток IndexOutOfRange НЕ викидається).
- **Трансляція в SQL:** У базах даних транслюється у вираз OFFSET n ROWS.

Результат для списку:

Після пропуску перших 2 рядків ('One', 'Two') повертаються: Three, Four, Five.

```
IList<string> strList = new List<string>() {
    "One", "Two", "Three", "Four", "Five"
};

// Пропускаємо перші 2 елементи колекції
var newList = strList.Skip(2);

Console.WriteLine("--- Результат Skip(2) ---");
foreach (var str in newList)
    Console.WriteLine(str);

// Пропуск більше ніж є елементів (порожній результат):
var empty = strList.Skip(10);
Console.WriteLine("Кількість у empty: " + empty.Count());

-> Output:
-> Three
-> Four
-> Five
-> Кількість у empty: 0
```

Program.cs

Ефективність:

Skip виконується відкладено і не матеріалізує пропущені елементи в оперативну пам'ять.

Оператор SkipWhile: Пропуск за предикатною умовою

Пропуск елементів до першого false

- **Предикатна перевірка:** SkipWhile обчислює умову s.Length < 4 для кожного елемента послідовно.
- **Елементи 1 і 2:** 'One' (довжина 3 < 4: true -> пропуск), 'Two' (довжина 3 < 4: true -> пропуск).
- **Точка зупинки:** На елементі 'Three' довжина дорівнює 5 (5 < 4: false). Пропуск негайно зупиняється!
- **Решта колекції:** Усі наступні елементи ('Four', 'Five', 'Six') повертаються без повторної перевірки умови.

Результат вибірки:

Повертаються: Three, Four, Five, Six.

```
IList<string> strList = new List<string>() {
    "One", "Two", "Three", "Four", "Five", "Six"
};

// Пропускаємо, поки довжина слова менша за 4
var resultList = strList.SkipWhile(s => s.Length < 4);

foreach (string str in resultList)
    Console.WriteLine(str);

-> Output:
-> Three
-> Four
-> Five
-> Six
```

Program.cs

Різниця з Where:

Where відфільтрував би й слово 'Six' (довжина 3), а SkipWhile повертає 'Six', бо після першого false перевірка припинилася!

SkipWhile: Поведінка, коли найперший елемент не задовольняє умову

Миттєве зупинення пропуску на позиції 0

- **Аналіз першого елемента:** Перший елемент списку — 'Three' (його довжина 5).
- **Умова s.Length < 4:** 5 < 4 одразу дає значення false!
- **Результат пропуску:** Оскільки перший же елемент не задовольнив умову, пропускається рівно 0 елементів.
- **Повернення даних:** Повертається абсолютно вся послідовність у повному обсязі, починаючи з найпершого елемента.

Результат вибірки:

Three, One, Two, Four, Five, Six — жоден елемент не пропущено!

```
IList<string> strList = new List<string>() {
    "Three", "One", "Two", "Four", "Five", "Six"
};

// Перший елемент 'Three' має довжину 5 >= 4
// Умова s.Length < 4 дає false одразу:
var resultList = strList.SkipWhile(s => s.Length < 4);

foreach (string str in resultList)
    Console.WriteLine(str);

-> Output:
-> Three
-> One
-> Two
-> Four
-> Five
-> Six
```

Program.cs

Головний висновок:

SkipWhile не шукає перше співпадіння вглибині колекції — він перевіряє елементи виключно від самого початку.

Оператор SkipWhile з використанням індексу елемента

Лямбда з парою (елемент, індекс)

- **Перевантаження з індексом:** SkipWhile((s, i) => s.Length > i) передає 0-based індекс кожного елемента.
- **Трасування кроків:** [0] 'One' (3 > 0: true), [1] 'Two' (3 > 1: true), [2] 'Three' (5 > 2: true), [3] 'Four' (4 > 3: true).
- **Критична точка [4]:** Елемент 'Five' (довжина 4): умова 4 > 4 стає false!
- **Зупинка пропуску:** Починаючи з індексу 4 ('Five') і до кінця списку, всі елементи повертаються.

Результат вибірки:

Повертаються лише останні елементи: Five та Six.

```
IList<string> strList = new List<string>() {
    "One", "Two", "Three", "Four", "Five", "Six"
};

// Пропускаємо, поки довжина рядка більша за його індекс:
// [0]'One'(3>0:T), [1]'Two'(3>1:T), [2]'Three'(5>2:T),
[3]'Four'(4>3:T)
// [4]'Five'(4>4: false -> зупинка пропуску!)
var result = strList.SkipWhile((s, i) => s.Length > i);

Console.WriteLine("--- Елементи після зупинки пропуску ---");
foreach (string str in result)
    Console.WriteLine(str);

-> Output:
-> Five
-> Six
```

Program.cs

Застосування індексів:

Індексовані предикати дозволяють реалізовувати алгоритми нелінійного ковзного вікна або складного позиційного розбору.

Оператор Take: Отримання перших N елементів послідовності

Вибірка фіксованої кількості елементів

- **Принцип роботи:** Take(count) повертає рівно count перших елементів від початку і негайно завершує перебір.
- **Раннє завершення:** Метод не сканує всю колекцію до кінця, що критично важливо для швидкодії.
- **Якщо count <= 0:** Повертається порожня послідовність.
- **Трансляція в SQL:** У базах даних транслятор LINQ генерує TOP (count) або FETCH NEXT count ROWS ONLY.

Результат для списку:

Take(2) повертає перші два елементи: One та Two.

```
IList<string> strList = new List<string>() {
    "One", "Two", "Three", "Four", "Five"
};

// Отримуємо рівно 2 перших елементи послідовності
var result = strList.Take(2);

Console.WriteLine("--- Перші 2 елементи ---");
foreach (string str in result)
    Console.WriteLine(str);

// Запит більшої кількості ніж є в наявності:
var all = strList.Take(10);
Console.WriteLine("Всього взято: " + all.Count());

-> Output:
-> One
-> Two
-> Всього взято: 5
```

Program.cs

Ранній вихід:

Take негайно зупиняє ітератор MoveNext() після отримання потрібної кількості елементів.

Оператор TakeWhile: Вибірка елементів до першого порушення умови

Отримання елементів, поки умова true

- **Принцип TakeWhile:** Бере елементи від початку списку, поки предикатна умова s.Length > 4 повертає true.
- **Крок 1 'Three':** Довжина 5 > 4: true -> елемент береться.
- **Крок 2 'Four':** Довжина 4 > 4: false -> вибірка негайно припиняється!
- **Ігнорування решти:** Навіть якщо далі є елемент 'Hundred' з довжиною 7 (> 4), він ніколи не буде оброблений.

Результат вибірки:

Повертається лише один елемент: Three.

```
IList<string> strList = new List<string>() {
    "Three", "Four", "Five", "Hundred"
};

// Беремо елементи, поки їх довжина більша за 4:
// [0]'Three'(len=5 > 4: true) -> взято
// [1]'Four' (len=4 > 4: false) -> СТОП!
var result = strList.TakeWhile(s => s.Length > 4);

Console.WriteLine("--- Результат TakeWhile ---");
foreach (string str in result)
    Console.WriteLine(str);

// 'Hundred' (len=7) ніколи не буде прочитано!
-> Output:
-> Three
Program.cs
```

Чому 'Hundred' не потрапив у результат:

TakeWhile повністю зупиняє ітерування, як тільки умова порушилася на слові 'Four'. Для повної фільтрації використовуйте Where.

Оператор TakeWhile з використанням індексу елемента

Індексована зупинка вибірки

- **Умова s.Length > i:** Порівнює довжину поточного рядка з його порядковим індексом.
- **[0] 'One' (3 > 0):** true -> беремо
- **[1] 'Two' (3 > 1):** true -> беремо
- **[2] 'Three' (5 > 2):** true -> беремо
- **[3] 'Four' (4 > 3):** true -> беремо
- **[4] 'Five' (4 > 4):** false -> фінальна зупинка!

Результат вибірки:

Отримано перші 4 елементи: One, Two, Three, Four.

```
IList<string> strList = new List<string>() {
    "One", "Two", "Three", "Four", "Five", "Six"
};

// Беремо, поки довжина рядка більша за його індекс
var resultList = strList.TakeWhile((s, i) => s.Length > i);

foreach (string str in resultList)
    Console.WriteLine(str);

-> Output:
-> One
-> Two
-> Three
-> Four
```

Program.cs

Резюме блоку Partitioning:

Skip, SkipWhile, Take та TakeWhile забезпечують повний контроль над порційним позиційним зчитуванням послідовностей.

Оператори приведення та матеріалізації типів (Conversion Operators)

Метод LINQ	Категорія	Опис / Призначення	Що повертає
`AsEnumerable`	As-оператор	Приховує кастомну реалізацію провайдера, повертаючи IEnumerable<T>.	IEnumerable<T>
`AsQueryable`	As-оператор	Перетворює IEnumerable на IQueryable для емуляції провайдера запитів.	IQueryable<T>
`Cast<T>`	Cast-оператор	Приводить елементи нетипізованої колекції (ArrayList) до узагальненого типу T.	IEnumerable<T>
`OfType<T>`	Cast-оператор	Безпечно фільтрує колекцію, вибираючи лише елементи вказаного типу T.	IEnumerable<T>
`ToArray`	To-оператор	Негайно виконує запит і копіює всі елементи в одновимірний масив T[].	T[]
`ToList`	To-оператор	Негайно виконує запит і створює новий екземпляр List<T> у пам'яті.	List<T>
`ToDictionary`	To-оператор	Матеріалізує елементи в словник Dictionary<TKey, TValue> за ключем.	Dictionary<K, V>
`ToLookup`	To-оператор	Групує елементи в незмінну 1:N структуру Lookup<TKey, TElement>.	ILookup<K, E>

Три групи перетворень:
As-оператори змінюють інтерфейс доступу; Cast-оператори валідують типи; To-оператори примусово матеріалізують дані (Immediate Execution).

Оператори AsEnumerable та AsQueryable: Статичні та динамічні типи

Зміна інтерфейсу представлення колекції

- **AsEnumerable():** Переводить статичний тип колекції в IEnumerable<T>. Критично у LINQ to Entities: дозволяє примусово переключити подальшу частину запиту на виконання в пам'яті C# замість SQL сервера.
- **AsQueryable():** Огортає звичайний IEnumerable у EnumerableQuery<T>, дозволяючи імітувати роботу IQueryable провайдера (наприклад, у Unit-тестах).
- **Типи компіляції та виконання:** Демонстрація різниці між typeof(T).Name та obj.GetType().Name під час виклику утиліти.
- **Клієнтська оцінка (Client Evaluation):** AsEnumerable перемикає виконання запиту в пам'ять процесу для виклику методів C#, які не підтримуються SQL.

Важливий нюанс:

AsEnumerable НЕ створює копії масиву — у рантаймі об'єкт залишається тим самим вихідним Student[].

```
class Program {
    static void ReportTypeProperties<T>(T obj) {
        Console.WriteLine("Compile-time: " + typeof(T).Name);
        Console.WriteLine("Actual runtime: " + obj.GetType().Name);
    }
    static void Main(string[] args) {
        Student[] studentArray = {
            new Student() { StudentID = 1, StudentName = "John", Age =
18 },
            new Student() { StudentID = 2, StudentName = "Steve", Age
= 21 }
        };
        ReportTypeProperties(studentArray);
        ReportTypeProperties(studentArray.AsEnumerable());
        ReportTypeProperties(studentArray.AsQueryable());
    }
}
-> Compile-time: Student[]     | Actual: Student[]
-> Compile-time: IEnumerable`1 | Actual: Student[]
-> Compile-time: IQueryable`1  | Actual: EnumerableQuery`1
```

Program.cs

Застосування AsEnumerable в EF Core:

Використовуйте AsEnumerable(), коли метод не може бути трансльований у SQL і його потрібно виконати на боці клієнта.

Оператор приведення Cast<T>: Робота з нетипізованими колекціями

Приведення типів елементів у послідовності

- **Призначення Cast<T>:** Перетворює непараметризовану колекцію (IEnumerable з .NET 1.1, наприклад ArrayList) на типізовану IEnumerable<T>.
- **Приведення кожного елемента:** Виконує явне приведення (T)element для кожного запису послідовності.
- **Ризик винятку:** Якщо хоча б один елемент у колекції має несумісний тип, виникає виняток InvalidCastException.
- **Безпечна альтернатива:** Якщо колекція містить різнорідні об'єкти, використовуйте OfType<T>(), який просто ігнорує несумісні типи.

Результат приведення:

Отримуємо строго типізований IEnumerable<Student>, з яким можна працювати за допомогою сучасного LINQ.

```
// Нетипізована колекція legacy .NET 1.1
System.Collections.ArrayList list = new
System.Collections.ArrayList();
list.Add(new Student() { StudentID = 1, StudentName = "John", Age =
18 });
list.Add(new Student() { StudentID = 2, StudentName = "Steve", Age
= 21 });
list.Add(new Student() { StudentID = 3, StudentName = "Bill", Age =
25 });

// Приведення до IEnumerable<Student>
var studentEnumerable = list.Cast<Student>();

foreach (Student std in studentEnumerable)
    Console.WriteLine(std.StudentName);

-> Output:
-> John
-> Steve
```

Відмінність від OfType:

Cast вимагає строгої відповідності 100% елементів типу T, тоді як OfType безпечно фільтрує невідповідні елементи без винятків.

Оператори матеріалізації: ToArray та ToList

Фіксація результатів запиту в оперативній пам'яті

- **Негайне виконання:** ToArray та ToList примусово виконують запит у момент їх виклику (Immediate Execution).
- **ToArray():** Створює одновимірний масив фіксованої довжини T[]. Ідеально для передачі в API з мінімальним оверхедом.
- **ToList():** Створює динамічний список List<T>, що дозволяє додавати та видаляти елементи.
- **Кешування результату:** Захищає від повторного виконання дорогих SQL-запитів при багаторазовому читанні даних.

Результат перетворення:

Отримуємо масив string[] або список List<string> з повним набором даних у пам'яті.

```
IList<string> strList = new List<string>() {
    "One", "Two", "Three", "Four", "Three"
};

// Перетворення списку в одновимірний масив
string[] strArray = strList.ToArray();

// Перетворення масиву назад у типізований список
IList<string> list = strArray.ToList();

Console.WriteLine("Масив: " + strArray.Length + " ел.");
Console.WriteLine("Список: " + list.Count + " ел.");

-> Output:
-> Масив: 5 ел.
-> Список: 5 ел.
Program.cs
```

Золоте правило:

Завжди матеріалізуйте дані через .ToList() перед закриттям з'єднання з базою даних або передачею моделі у представлення.

Оператор ToDictionary: Створення асоціативної таблиці за ключем

Матеріалізація в Dictionary<TKey, TValue>

- **Селектор ключа:** ToDictionary вимагає обов'язкової функції вибору ключа, наприклад s => s.StudentID.
- **Швидкий доступ O(1):** Словник забезпечує миттєвий пошук студента за його ID без необхідності перебору колекції.
- **Вимога унікальності:** Усі ключі повинні бути суворо унікальними! Якщо трапиться два однакових StudentID, метод викине ArgumentException.
- **Селектор значення:** Можна опційно передати другу лямбду для збереження лише окремого поля: s => s.StudentName.

Результат матеріалізації:

Створено Dictionary<int, Student>, де ключем є StudentID, а значенням — об'єкт студента.

```
IList<Student> studentList = new List<Student>() {
    new Student() { StudentID = 1, StudentName = "John", Age = 18 },
    new Student() { StudentID = 2, StudentName = "Steve", Age = 21 },
    new Student() { StudentID = 3, StudentName = "Bill", Age = 18 },
    new Student() { StudentID = 4, StudentName = "Ram", Age = 20 },
    new Student() { StudentID = 5, StudentName = "Ron", Age = 21 }
};

// Конвертація в Dictionary з ключем StudentID
IDictionary<int, Student> studentDict = studentList
    .ToDictionary<Student, int>(s => s.StudentID);

foreach (var key in studentDict.Keys)
    Console.WriteLine($"Key: {key}, Value:
{studentDict[key].StudentName}");

-> Key: 1, Value: John
-> Key: 2, Value: Steve
-> Key: 3, Value: Bill...
```

Program.cs

Що робити з дублікатами ключів:

Якщо ключі можуть повторюватися, замість ToDictionary використовуйте ToLookup() або GroupBy(), які підтримують групи 1:N.

Відкладене виконання (Deferred Execution) у LINQ

Концепція відкладеного обчислення

- **Що таке Deferred Execution:** Оголошення LINQ-запиту НЕ виконує сам запит і не перебирає елементи колекції в пам'яті.
- **Збереження плану запиту:** Змінна запиту teenAgerStudents зберігає лише команду та логіку обробки (структуру IEnumerable).
- **Точка виконання:** Запит реально виконується тільки тоді, коли дані безпосередньо запитуються (наприклад, у циклі foreach).
- **Оптимізація ресурсів:** Дозволяє уникати непотрібних проміжних обчислень та об'єднувати фільтри перед відправкою до бази даних.

Дві критичні точки в коді:

1. Оголошення запиту: нуль операцій з елементами. 2. Цикл foreach: послідовне виконання ітератора.

```
IList<Student> studentList = new List<Student>() {
    new Student() { StudentID = 1, StudentName = "John", Age = 18 },
    new Student() { StudentID = 2, StudentName = "Steve", Age = 21 },
    new Student() { StudentID = 3, StudentName = "Bill", Age = 18 },
    new Student() { StudentID = 4, StudentName = "Ram", Age = 20 },
    new Student() { StudentID = 5, StudentName = "Ron", Age = 21 }
};

// ТОЧКА 1: Запит ТУТ НЕ ВИКОНУЄТЬСЯ!
var teenAgerStudents = from s in studentList
                       where s.Age > 12 && s.Age < 20
                       select s;

// ТОЧКА 2: Запит РЕАЛЬНО ВИКОНУЄТЬСЯ ТУТ!
foreach (Student teenStudent in teenAgerStudents)
    Console.WriteLine("Name: " + teenStudent.StudentName);

-> Output:
-> Name: John
-> Name: Bill
```

Program.cs

Універсальність:

Deferred execution діє як для локальних колекцій у пам'яті (LINQ to Objects), так і для віддалених джерел (LINQ to Entities, SQL, XML).

Повторне обчислення запиту (Query Re-evaluation)

Динамічний доступ до актуальних даних

- **Відсутність кешу за замовчуванням:** Оскільки відкладений запит не зберігає результатів у пам'яті, кожен новий цикл foreach звертається до джерела наново.
- **Модифікація колекції:** Якщо після першого виконання додати нового студента (Martin, 16 років), він автоматично врахується при повторному обході!
- **Один запит — актуальні дані:** Запит можна оголосити один раз і повторно використовувати його для отримання найсвіжіших даних з джерела.
- **Уникнення подвійного обчислення:** Якщо джерелом є база даних або складний розрахунок, багаторазовий перехід запиту виконує повторні мережеві запити. Фіксуйте результат через .ToList().

Результати двох циклів:

Перший foreach: John, Bill. Другий foreach (після Add): John, Bill, Martin!

```
// Перше виконання запиту:
foreach (Student teenStudent in teenAgerStudents)
    Console.WriteLine("Name: " + teenStudent.StudentName);
// -> John, Bill

// Додаємо нового студента у вихідний список:
studentList.Add(new Student() {
    StudentID = 6, StudentName = "Martin", Age = 16
});

// Друге виконання того ж самого запиту повертає свіжі дані:
foreach (Student teenStudent in teenAgerStudents)
    Console.WriteLine("Name: " + teenStudent.StudentName);

-> Output другого циклу:
-> Name: John
-> Name: Bill
-> Name: Martin (доданий елемент враховано!)
```

Program.cs

Важливий побічний ефект:

Якщо колекція змінюється між ітераціями, результати будуть різними. Якщо потрібен незмінний знімок даних — використовуйте .ToList().

Реалізація відкладеного виконання: Механізм yield return

Як побудовані ітератори BCL зсередини

- **Ключове слово yield return:** Перетворює звичайний C# метод на генератор ітератора, що реалізує інтерфейс IEnumerator<T>.
- **Скінченний автомат:** Компілятор генерує прихований клас State Machine, який зберігає стан виконання між викликами MoveNext().
- **Потокова обробка:** Метод не повертає всю колекцію відразу — він повертає рівно один елемент і призупиняється до наступного запиту.
- **Очищення ресурсів (Dispose):** Ітератор гарантує виклик блоків finally та Dispose, коли цикл foreach завершується або переривається за допомогою break.

Власний LINQ оператор:

Метод GetTeenAgerStudents фільтрує студентів потоково, логуючи кожне звернення до елемента.

```
public static class EnumerableExtensionMethods {
    // Кастомний метод-розширення з відкладеним виконанням
    public static IEnumerable<Student> GetTeenAgerStudents(
        this IEnumerable<Student> source)
    {
        foreach (Student std in source)
        {
            Console.WriteLine("Accessing student {0}",
std.StudentName);

            if (std.Age > 12 && std.Age < 20)
            {
                yield return std; // пауза виконання до наступного
MoveNext()
            }
        }
    }
}
```

EnumerableExtensionMethods.cs

Магія yield return:

Код методу взагалі не виконується в момент виклику GetTeenAgerStudents() — він стартує тільки при першому зверненні до ітератора!

Трасування роботи ітератора з yield return на практиці

Доведення відкладеності у виводі консолі

- **Крок 1 (Оголошення):** Виклик GetTeenAgerStudents() у секції from s in ... не генерує ЖОДНОГО повідомлення в консолі!
- **Крок 2 (Ітерація):** У циклі foreach починає викликатися MoveNext(). Метод виконується поетапно для кожного студента.
- **Аналіз виводу:** Рядки 'Accessing student...' чергуються з виведенням імені в циклі 'Name: ...'.
- **Ранній вихід:** Якби в циклі стояв break після першого студента, перевірка решти списку навіть не розпочалася б.

Результат трасування:

Повний синхронний контроль над споживанням даних без виділення проміжних масивів.

```
// 1. Оголошення запиту - GetTeenAgerStudents НЕ викликається:
var teenAgerStudents = from s in studentList.GetTeenAgerStudents()
                       select s;

Console.WriteLine("Запит оголошено. Починаємо foreach:");

// 2. Ітерація - ТІЛЬКИ ТУТ починається обробка елементів:
foreach (Student teenStudent in teenAgerStudents)
{
    Console.WriteLine("-> Знайдено: " + teenStudent.StudentName);
}

-> Console Trace:
-> Accessing student John -> Знайдено: John
-> Accessing student Steve
-> Accessing student Bill -> Знайдено: Bill
-> Accessing student Ram
-> Accessing student Ron
```

Program.cs

Підсумок механізму:

Це фундаментальна основа функціонального програмування в C#: потокова конвеєризація даних (Pipeline Processing).

Негайне виконання запитів (Immediate Execution)

Примусове обчислення в точці виклику

- **Що таке Immediate Execution:** Повна протилежність відкладеному виконанню. Змушує запит негайно обробити всі дані і зберегти знімок результату.
- **Оператори негайного виконання:** Всі 'To' оператори (ToList, ToArray, ToDictionary, ToLookup), а також агрегатні функції (Count, Average, First, Single, Max).
- **Ізоляція від змін:** Результат зберігається у створеній колекції (List<Student>), тому подальші модифікації studentList уже не впливають на збережені дані.
- **Використання пам'яті:** Матеріалізація завантажує всю вибірку в RAM, тому для великих обсягів даних слід обмежувати вибірку через Take().

Приклад у Method Syntax:

Метод .ToList() негайно фільтрує список і повертає готовий об'єкт List<Student>.

```
// Негайне виконання за допомогою .ToList():
IList<Student> teenAgerStudents = studentList
    .Where(s => s.Age > 12 && s.Age < 20)
    .ToList();

// Запит повністю виконано прямо зараз у точці .ToList()!
// Змінна teenAgerStudents вже містить 2 об'єкти в пам'яті.
Console.WriteLine($"Завантажено студентів: {teenAgerStudents.Count}");

foreach (Student teenStudent in teenAgerStudents)
{
    Console.WriteLine($"ID: {teenStudent.StudentID}, Name:
{teenStudent.StudentName}");
}

-> Output:
-> Завантажено студентів: 2
-> ID: 1, Name: John
-> ID: 3, Name: Bill
```

Program.cs

Коли застосовувати Immediate Execution:

Завжди, коли потрібно зафіксувати дані в пам'яті (кеш), від'єднатися від БД або запобігти повторним важким обрахункам.

Негайне виконання у синтаксисі запиту (Query Syntax)

Керування виконанням через круглі дужки

- **Природа Query Syntax:** Синтаксис запиту (from ... select) за замовчуванням ЗАВЖДИ є відкладеним (повертає IEnumerable).
- **Прийом обгортання:** Щоб виконати такий запит негайно, весь вираз береться в круглі дужки і викликається оператор матеріалізації: (...) .ToList().
- **Перегляд у Debugger:** У відкладеному запиті налагоджувач показує внутрішній тип ітератора. У матеріалізованому — одразу відображає Count = 2 та масив елементів.
- **Матеріалізація скалярів:** Будь-який запит Query Syntax можна також обгорнути в дужки для виклику First(), Count(), Any() або ToDictionary().

Практична різниця:

Без .ToList(): запит чекає ітерації. З .ToList(): у пам'яті одразу створено готовий список із 2 студентів.

```
// 1. Відкладений синтаксис запиту:
var deferredQuery = from s in studentList
                    where s.Age > 12 && s.Age < 20
                    select s;

// 2. Негайне виконання через круглі дужки:
IList<Student> teenAgerStudents = (
    from s in studentList
    where s.Age > 12 && s.Age < 20
    select s
).ToList();

// QuickWatch / Debugger View:
// teenAgerStudents.Count = 2
// [0] = {StudentID = 1, StudentName = "John"}
// [1] = {StudentID = 3, StudentName = "Bill"}
```

Program.cs

Патерн для команди:

Обгортання запиту в дужки з викликом .ToList() або .ToArray() є загальноприйнятим стандартом у комерційній розробці на C#.

Ключове слово let: Оголошення проміжних змінних у запиті

Оптимізація та читабельність LINQ запитів

- **Проблема без let:** Якщо потрібно використовувати один і той самий вираз (наприклад, s.StudentName.ToLower()) у where та select, його доводиться писати двічі.
- **Призначення let:** Ключове слово let вводить нову змінну в запиті та зберігає в ній проміжний результат обчислення.
- **Прозорий ідентифікатор:** Компілятор C# створює анонімний тип (Transparent Identifier), що містить вихідний об'єкт та нову змінну.
- **Підвищення продуктивності:** Метод ToLower() викликається лише один раз для кожного запису замість двох чи більше разів.

Результат виконання:

Вибірка містить імена в нижньому регістрі: 'ram', 'ron'.

```
// БЕЗ LET: ToLower() викликається двічі для кожного студента
var badQuery = from s in studentList
               where s.StudentName.ToLower().StartsWith("r")
               select s.StudentName.ToLower();

// З LET: обчислюємо один раз і повторно використовуємо
var lowercaseStudentNames = from s in studentList
    let lowercaseName = s.StudentName.ToLower()
    where lowercaseName.StartsWith("r")
    select lowercaseName;

foreach (var name in lowercaseStudentNames)
    Console.WriteLine(name);

-> Output:
-> ram
-> ron
Program.cs
```

Best Practice:

Використовуйте let для збереження складних математичних формул, викликів методів та розбору рядків усередині запиту.

Ключове слово into: Продовження запиту (Query Continuation)

Конвеєризація запиту після select або group

- **Призначення into:** Дозволяє продовжити виконання запиту після секцій select, group by або join, створюючи новий ідентифікатор діапазону.
- **Область видимості:** Увага! Після ключового слова into попередня змінна діапазону (s) виходить з області видимості і більше недоступна!
- **Багаторівнева фільтрація:** У наведеному прикладі ми спочатку вибираємо підлітків, а потім продовжуємо фільтрацію за першою літерою імені ('B').
- **Завершення курсу LINQ:** Повне поєднання Query та Method синтаксису дає найпотужніший інструмент роботи з даними у .NET 10.

Фінальний результат запиту:

Повертається студент Bill, який пройшов обидва етапи фільтрації.

```
// Продовження запиту після select за допомогою 'into'
var teenAgerStudents =
    from s in studentList
    where s.Age > 12 && s.Age < 20
    select s
    into teenStudents // 's' більше недоступна, працюємо з
teenStudents
    where teenStudents.StudentName.StartsWith("B")
    select teenStudents;

Console.WriteLine("--- Студенти після into-фільтрації ---");
foreach (var teen in teenAgerStudents)
{
    Console.WriteLine($"ID: {teen.StudentID}, Name:
{teen.StudentName}, Age: {teen.Age}");
}

-> Output:
-> ID: 3, Name: Bill, Age: 18
```

Program.cs

Вітаємо із завершенням курсу!

Ви опанували всі 18 ключових тем LINQ: від базової архітектури та лямбд до складних реляційних з'єднань, секціонування та оптимізації виконання!